

Universidad Politécnica de Madrid

[COMPLETAR: Escuela Técnica Superior de ...]

[COMPLETAR: Grado en Ingeniería ...]

TRABAJO FIN DE GRADO

Caracola de radio variable accionada por servomotor paso a paso

Diseño del sistema electrónico de control y adquisición
y validación experimental de la compensación de par

Autoría: Javier López [COMPLETAR: segundo apellido]

Tutoría: [COMPLETAR: nombre del tutor o tutora]

Fecha: [COMPLETAR: mes] de 2026

Agradecimientos

[**COMPLETAR:** Texto de agradecimientos: tutoría, laboratorio o departamento que ha prestado el material, familia, compañeros y compañeras.]

Resumen

Este Trabajo Fin de Grado aborda el diseño, la construcción y la validación experimental del sistema electrónico de accionamiento, control y adquisición de un mecanismo de elevación basado en una *caracola*: un tambor de radio variable que, movido por un servomotor paso a paso de lazo cerrado, enrolla un cable que levanta una barra articulada con una masa de 2 kg. La finalidad de la caracola es modificar la relación de transmisión a lo largo del recorrido para que el par que debe aportar el motor se mantenga aproximadamente constante, aunque la tensión del cable varíe con la geometría del mecanismo.

En primer lugar se desarrolla un modelo analítico del mecanismo. Se demuestra que la tensión del cable es proporcional a la longitud libre de cable, $T = mg(L_3 + L_4) c / (L_2 L_3)$, y se deduce el perfil de radio que hace el par exactamente constante, $r(\varphi) = C / \sqrt{c_0^2 + 2C\varphi}$. Se prueba además que ese par constante, igual al trabajo total dividido por el giro del tambor, es el mínimo par de pico alcanzable con cualquier perfil. Con la geometría nominal del prototipo, el perfil ideal reduce el par de pico un 24,7 % respecto a un tambor cilíndrico que haga el mismo recorrido, y una espiral de radio lineal optimizada lo reduce un 21,1 %.

Para verificarlo experimentalmente se ha construido una plataforma completa: una placa adaptadora diseñada en KiCad alrededor de una Raspberry Pi Pico 2 (bus RS-485, entradas digitales protegidas, entradas analógicas acondicionadas, salidas de potencia y dos zócalos mikroBUS para células de carga); un firmware en C++ que gobierna el servo mediante Modbus RTU, lee la célula de carga a través del acondicionador ZSC31014, realiza el *homing* con dos finales de carrera y ejecuta ciclos automáticos; un protocolo binario propio con CRC-8; y una aplicación web (FastAPI y WebSocket) para la supervisión en tiempo real, el registro de ensayos y su comparación con el modelo.

En los ensayos, el sistema muestrea a 20,0 Hz con una dispersión del periodo de 0,07 ms, alcanza el final de carrera con una desviación típica de 13 mrev y vuelve al origen con un error máximo de 17 mrev. Separando en el par medido la componente gravitatoria y la de pérdidas mediante recorridos en ambos sentidos, se observa que en el tramo central del recorrido la tensión del cable crece un 17 % mientras que el par gravitatorio del motor varía sólo un 6,4 %, lo que confirma experimentalmente el efecto compensador de la caracola.

Palabras clave: tambor de radio variable, caracola, compensación de par, servomotor paso a paso, Modbus RTU, RS-485, célula de carga, ZSC31014, Raspberry Pi Pico 2, adquisición de datos, FastAPI.

Abstract

This Bachelor's Thesis covers the design, construction and experimental validation of the electronic drive, control and data-acquisition system for a lifting mechanism based on a *fusee-like spiral drum* ("caracola"): a variable-radius drum, driven by a closed-loop stepper servomotor, winds a cable that lifts a hinged bar carrying a 2 kg mass. The purpose of the spiral drum is to change the transmission ratio along the stroke so that the motor torque remains approximately constant while the cable tension changes with the geometry.

An analytical model of the mechanism is first derived. The cable tension is shown to be proportional to the free cable length, $T = mg(L_3 + L_4) c / (L_2 L_3)$, and the radius profile that makes the torque exactly constant, $r(\varphi) = C / \sqrt{c_0^2 + 2C\varphi}$, is obtained. This constant torque, equal to the total work divided by the drum rotation, is proven to be the lowest peak torque achievable by any profile. With the nominal geometry, the ideal profile lowers the peak torque by 24,7 % compared with a cylindrical drum covering the same stroke, and an optimised linear spiral lowers it by 21,1 %.

To verify this experimentally, a complete platform was built: a KiCad-designed adapter board around a Raspberry Pi Pico 2 (RS-485 bus, protected digital inputs, conditioned analogue inputs, high-side outputs and two mikroBUS sockets for load cells); C++ firmware that commands the servo over Modbus RTU, reads the load cell through a ZSC31014 conditioner, performs homing with two limit switches and runs automatic cycles; a custom binary protocol with CRC-8; and a web application (FastAPI and WebSocket) for real-time monitoring, test logging and comparison with the model.

In the tests, the system samples at 20,0 Hz with a period spread of 0,07 ms, reaches the limit switch with a standard deviation of 13 mrev and returns home with a maximum error of 17 mrev. By separating the gravitational and loss components of the measured torque using strokes in both directions, it is observed that over the central part of the stroke the cable tension rises by 17 % while the gravitational motor torque varies by only 6,4 %, experimentally confirming the compensating effect of the spiral drum.

Keywords: variable radius drum, fusee, torque compensation, closed-loop stepper servo, Modbus RTU, RS-485, load cell, ZSC31014, Raspberry Pi Pico 2, data acquisition, FastAPI.

Índice general

Agradecimientos	I
Resumen	III
Abstract	V
Acrónimos y símbolos	XVII
1. Introducción	1
1.1. Contexto y motivación	1
1.2. Objetivos	1
1.2.1. Objetivo general	1
1.2.2. Objetivos específicos	1
1.3. Alcance y limitaciones	2
1.4. Metodología	2
1.5. Estructura del documento	3
2. Estado del arte y fundamentos	5
2.1. Mecanismos de transmisión con radio variable	5
2.1.1. Del <i>fusee</i> a los tambores de radio variable	5
2.1.2. Compensación de gravedad con poleas no circulares	5
2.2. Accionamiento: servomotores paso a paso de lazo cerrado	6
2.3. Medida de fuerza con células de carga	6
2.3.1. Galgas extensométricas y puente de Wheatstone	6
2.3.2. Acondicionadores integrados	6
2.4. Microcontrolador	7
2.5. Buses de comunicación	7
2.5.1. RS-485 y Modbus RTU	7
2.5.2. I ² C	7
2.6. Supervisión y adquisición en el ordenador	8
2.7. Conclusiones del estado del arte	8
3. Modelo del mecanismo	9
3.1. Descripción del mecanismo	9
3.2. Longitud del cable	10
3.3. Tensión del cable	10
3.4. Par en el tambor	11
3.4.1. Cota inferior del par de pico	11
3.4.2. Tambor cilíndrico	12

3.4.3.	Perfil ideal de par constante	12
3.4.4.	Espiral de radio lineal	12
3.4.5.	Comparación	12
3.5.	Separación experimental de las componentes del par	13
3.6.	Limitaciones del modelo	14
4.	Requisitos y arquitectura del sistema	15
4.1.	Requisitos	15
4.1.1.	Requisitos funcionales	15
4.1.2.	Requisitos no funcionales	15
4.2.	Arquitectura general	16
4.3.	Flujo de datos y temporización	17
4.4.	Decisiones de diseño	17
4.5.	Seguridad	18
5.	Diseño hardware	21
5.1.	Placa adaptadora: objetivos	21
5.2.	Revisiones del diseño	21
5.3.	Microcontrolador	22
5.4.	Interfaz RS-485	22
5.5.	Entradas digitales	23
5.6.	Entradas analógicas	23
5.7.	Salidas de potencia	24
5.8.	Célula de carga	24
5.9.	Alimentación	24
5.10.	Placa de circuito impreso	24
5.11.	Validación de la placa	26
6.	Firmware del microcontrolador	29
6.1.	Entorno y organización	29
6.2.	Lazo principal	29
6.3.	Máquina de estados del servo	30
6.4.	Lectura de la célula de carga	31
6.4.1.	Árbitro del bus I ² C	31
6.5.	Posición integrada	31
6.6.	<i>Homing</i>	32
6.7.	Ciclo automático A↔B	32
6.8.	Configuración del acondicionador de la célula	33
6.8.1.	Escritura verificada	33
6.8.2.	Mantenimiento del reinicio para medida	34
6.9.	Evolución del firmware	34
7.	Comunicaciones	35
7.1.	Protocolo binario entre el microcontrolador y el ordenador	35
7.1.1.	Formato de trama	35
7.1.2.	Recepción y resincronización	36

7.1.3.	Paquetes del microcontrolador al ordenador	36
7.1.4.	Órdenes del ordenador al microcontrolador	37
7.2.	Modbus RTU con el accionamiento	38
7.2.1.	Registros utilizados	38
7.2.2.	Secuencia de configuración	38
7.2.3.	Transacciones por periodo	38
7.3.	Interfaz entre el servidor y el navegador	38
7.3.1.	WebSocket	38
7.3.2.	Servicios HTTP	39
8.	Software de supervisión y análisis	41
8.1.	Visión general	41
8.2.	Servidor	41
8.2.1.	Estructura concurrente	41
8.2.2.	Grabación de ensayos	42
8.3.	Página de supervisión	42
8.4.	Visor de ensayos	43
8.5.	Herramientas auxiliares	44
8.6.	Observaciones de calidad	44
9.	Problemas técnicos y soluciones	45
9.1.	UART sobre PIO y latencia del enlace Modbus	45
9.2.	Configuración del ZSC31014: la ventana del modo comando	46
9.2.1.	Primer obstáculo: provocar el reinicio	46
9.2.2.	Segundo obstáculo: alimentación parásita por el bus	46
9.2.3.	Tercer obstáculo: acertar la ventana	47
9.2.4.	Cuarto obstáculo: integridad de la EEPROM	47
9.3.	Lecturas de la célula siempre “antiguas”	47
9.4.	Discrepancias con la hoja de datos detectadas durante la redacción	48
9.5.	Segunda célula de carga en el mismo bus	48
9.6.	Otros problemas	49
9.7.	Lecciones aprendidas	49
10.	Resultados experimentales	51
10.1.	Condiciones de ensayo	51
10.2.	Temporización del lazo de control	52
10.3.	Ensayo tipo	53
10.4.	Repetibilidad de posición	53
10.5.	Tensión del cable y par del motor frente a la posición	53
10.6.	Separación de componentes y efecto de la caracola	55
10.7.	Influencia de la velocidad	58
10.8.	Coherencia con el modelo	58
10.9.	Limitaciones de las medidas	59
10.9.1.	Procedimiento de calibración propuesto	59

11. Planificación, presupuesto e impacto	61
11.1. Planificación	61
11.1.1. Fases	61
11.2. Presupuesto	62
11.2.1. Recursos humanos	62
11.2.2. Material	63
11.2.3. Equipos y software	64
11.2.4. Resumen	64
11.3. Impacto	64
11.3.1. Impacto técnico y económico	64
11.3.2. Objetivos de Desarrollo Sostenible	65
11.3.3. Aspectos éticos, legales y de seguridad	65
12. Conclusiones y trabajo futuro	67
12.1. Grado de cumplimiento de los objetivos	67
12.2. Conclusiones	67
12.3. Trabajo futuro	68
12.3.1. Medida y validación	68
12.3.2. Firmware y comunicaciones	68
12.3.3. Hardware y seguridad	69
12.4. Valoración personal	69
Bibliografía	71
A. Asignación de pines	73
A.1. Raspberry Pi Pico 2 (revisión B de la placa)	73
A.2. Bornas de campo	74
B. Especificación del protocolo binario	75
B.1. Paquete 0x02: telemetría por lotes	75
B.2. Paquete 0x03: estado de <i>homing</i> (11 bytes)	76
B.3. Paquete 0x04: resultado de la célula (13 bytes)	76
B.4. Paquete 0x05: estado del ciclo (14 bytes)	76
B.5. Órdenes	77
B.6. Tablas de códigos	77
C. Registros Modbus del accionamiento	79
D. Formato de los ficheros de ensayo	81
E. Manual de uso	83
E.1. Instalación	83
E.1.1. Firmware	83
E.1.2. Aplicación de supervisión	83
E.2. Procedimiento de ensayo	83
E.3. Programación de la célula de carga	84
E.4. Diagnóstico	84

E.5. Precauciones	84
F. Estructura del repositorio	85

Índice de figuras

3.1. Esquema del mecanismo de elevación con caracola.	9
3.2. Modelo con la geometría nominal. Izquierda: tensión del cable frente al ángulo de la barra. Centro: radio de enrollamiento de los tres perfiles. Derecha: par en el tambor.	13
4.1. Arquitectura del sistema de accionamiento, control y adquisición.	17
5.1. Esquema de la placa adaptadora, revisión B.	25
5.2. PCB de la revisión A: capas de cobre, máscara y serigrafía superiores.	25
5.3. Vista 3D de la PCB de la revisión A generada con KiCad.	26
6.1. Máquina de estados del enlace con el accionamiento.	30
6.2. Máquina de estados del <i>homing</i>	32
7.1. Formato común de trama. El CRC-8 cubre los LEN bytes de la carga útil (ID + datos).	35
8.1. Estructura del servidor de supervisión.	41
10.1. Izquierda: periodo entre muestras (escala logarítmica). Derecha: tiempo de trabajo por vuelta del lazo. Todos los ensayos válidos.	52
10.2. Ensayo tipo a 30 rpm: posición, velocidad, célula de carga (con la escala provisional en newtons a la derecha) y par del motor.	54
10.3. Repetibilidad en los 14 ciclos a 30 rpm. Izquierda: posición a la que se alcanza B. Derecha: posición final en A, con la banda de tolerancia de ± 20 mrev.	55
10.4. Tensión del cable (arriba) y par del motor (abajo) frente a la posición, en los 12 ensayos a 30 rpm. Azul: hacia B (baja la masa). Rojo: hacia A (sube la masa).	56
10.5. Arriba: tensión media por intervalo en cada sentido y su promedio. Centro: par en cada sentido, componente gravitatoria τ_g y de pérdidas τ_f . Abajo: radio efectivo relativo τ_g/F , normalizado a su máximo.	57
10.6. Par del motor frente a la posición a 20, 30 y 40 rpm. Línea continua: hacia B. Discontinua: hacia A.	58
11.1. <i>Commits</i> por semana en el repositorio del proyecto.	61
11.2. Diagrama de Gantt reconstruido (semanas desde el 2 de marzo de 2026).	62

Índice de tablas

2.1. Características del microcontrolador RP2350 en la Raspberry Pi Pico 2 [20]. .	7
3.1. Geometría nominal empleada en el modelo. Son los valores por defecto de la interfaz de supervisión [VERIFICAR: sustituir por las cotas medidas en el prototipo y regenerar las figuras]	11
3.2. Comparación de perfiles de tambor con la geometría nominal (recorrido completo, $\varphi_{\text{tot}} = 0,5 \text{ rev}$).	13
4.3. Decisiones de diseño y alternativas consideradas.	18
5.1. Problemas detectados en la revisión A y corrección en la revisión B.	22
5.2. Pruebas de la placa adaptadora.	26
6.1. Versiones del firmware conservadas en el repositorio.	34
7.1. Enlaces de comunicación del sistema.	35
7.2. Paquetes de telemetría y estado.	36
7.3. Órdenes aceptadas por el firmware.	37
7.4. Parámetros configurables con SET_PARAM.	37
7.5. Servicios HTTP del servidor.	39
10.1. Ensayos grabados. Velocidad de consigna, repeticiones, muestras, duración y posición máxima alcanzada (final de carrera B).	51
11.1. Dedicación estimada y coste de personal.	63
11.2. Material del sistema.	63
11.3. Amortización de equipos.	64
11.4. Presupuesto total.	64
11.5. Relación del trabajo con los Objetivos de Desarrollo Sostenible [19].	65
A.1. Asignación de GPIO según el esquema de la revisión B y el firmware.	73
A.2. Bornas de la placa (revisión B).	74
B.1. Cabecera del lote (14 bytes).	75
B.2. Muestra (14 bytes), repetida n veces.	75
B.3. Estados de la máquina.	77
B.4. Códigos de error.	77
B.5. Resultados de ModbusMaster.	77
B.6. Resultados de las órdenes de la célula.	78
C.1. Registros utilizados por el firmware.	79

D.1. Columnas del fichero de ensayo.	81
--	----

Acrónimos y símbolos

Acrónimos

Sigla	Significado
A/D, ADC	Convertidor analógico-digital (<i>Analog-to-Digital Converter</i>)
API	Interfaz de programación de aplicaciones
CDC	Clase de dispositivo USB de comunicaciones (puerto serie virtual)
CRC	Código de redundancia cíclica (<i>Cyclic Redundancy Check</i>)
CSV	Valores separados por comas (<i>Comma-Separated Values</i>)
DE / RE	Habilitación del emisor / receptor de un transceptor RS-485
DI / DO	Entrada digital / salida digital
EEPROM	Memoria no volátil borrrable eléctricamente
ESD	Descarga electrostática (<i>Electrostatic Discharge</i>)
FC	Final de carrera
GPIO	Entrada/salida de propósito general
GUI	Interfaz gráfica de usuario
HTTP	Protocolo de transferencia de hipertexto
I ² C	Bus serie síncrono de dos hilos (<i>Inter-Integrated Circuit</i>)
LC	Célula de carga (<i>Load Cell</i>); LC1 y LC2 en el firmware
MISR	Registro de firma de la EEPROM del ZSC31014 (<i>Multiple-Input Signature Register</i>)
NC	Contacto normalmente cerrado
ODS	Objetivos de Desarrollo Sostenible
PCB	Placa de circuito impreso (<i>Printed Circuit Board</i>)
PIO	E/S programable del RP2350 (<i>Programmable I/O</i>)
POR	Reinicio por encendido (<i>Power-On Reset</i>)
RPM, rpm	Revoluciones por minuto
RS-485	Estándar TIA/EIA-485 de transmisión diferencial multipunto
RTU	Modo binario de Modbus sobre línea serie (<i>Remote Terminal Unit</i>)
SMD	Componente de montaje superficial
STO	Desconexión segura del par (<i>Safe Torque Off</i>)
TFG	Trabajo Fin de Grado
UART	Transmisor-receptor asíncrono universal
USB	Bus serie universal
VRD	Tambor de radio variable (<i>Variable Radius Drum</i>)
WS	WebSocket

Símbolos principales

Símbolo	Descripción	Unidad
L_2	Altura del punto de salida del cable sobre la articulación O	mm
L_3	Distancia de O al punto de anclaje del cable en la barra	mm
L_4	Distancia del anclaje del cable al punto de suspensión de la masa	mm
m	Masa suspendida	kg
g	Aceleración de la gravedad	m/s ²
θ	Ángulo de la barra respecto a la horizontal (positivo hacia arriba)	°
c	Longitud libre de cable entre la salida y el anclaje	mm
T	Tensión del cable	N
K	Constante de proporcionalidad $T = Kc$	N/mm
φ	Giro del tambor (caracola) medido desde el origen del recorrido	rad
$r(\varphi)$	Radio instantáneo de enrollamiento del tambor	mm
$s(\varphi)$	Cable pagado por el tambor, $s = \int r \, d\varphi$	mm
τ	Par en el eje del tambor	N · m
N	Reducción motor:tambor (vueltas de motor por vuelta de tambor)	—
τ_g, τ_f	Componentes gravitatoria y de pérdidas del par medido	u. a.
p	Posición integrada del eje del motor	rev
u. a.	Unidades internas del accionamiento (registro 16387)	—

Capítulo 1

Introducción

1.1. Contexto y motivación

Muchos mecanismos de elevación no presentan una carga constante al actuador. En una barra articulada que se levanta tirando de un cable, la tensión que hace falta para sostener la carga depende del ángulo que forman el cable y la barra; al girar la barra, esa geometría cambia y con ella la tensión. Si el cable se enrolla en un tambor cilíndrico, el par que el motor debe aportar, producto de la tensión por el radio, sigue fielmente esas variaciones. El actuador se tiene que dimensionar entonces para el peor punto del recorrido, aunque la mayor parte del tiempo trabaje muy por debajo de él.

Una solución clásica es cambiar la relación de transmisión a lo largo del recorrido haciendo que el radio del tambor no sea constante. Es la idea del *fusee* de la relojería del siglo XV, un cono con una ranura helicoidal que compensaba la pérdida de par del muelle real a medida que se desenrollaba [1]. La misma idea reaparece hoy en los tambores de radio variable de la robótica de cables [3, 4] y en los sistemas pasivos de compensación de gravedad [5, 6]. En este trabajo, al tambor en espiral se le llama *caracola* por su forma.

El proyecto en que se enmarca este TFG consiste en una caracola, accionada por un servomotor paso a paso de lazo cerrado, que enrolla un cable y levanta una barra articulada con una masa de 2 kg. Lo que se quiere demostrar es que el perfil de radio variable de la caracola aplanará el par que ve el motor frente a la variación de la tensión en el cable. Demostrarlo exige dos cosas: un modelo que diga qué cabe esperar y un sistema de medida capaz de registrar a la vez la posición del eje, la tensión del cable y el par del motor en ensayos repetibles. Construir ese sistema ha sido la mayor parte del trabajo.

1.2. Objetivos

1.2.1. Objetivo general

Diseñar, construir y validar un sistema electrónico de accionamiento, control y adquisición para un mecanismo de elevación con caracola de radio variable, y utilizarlo para comprobar experimentalmente que la caracola reduce la variación del par motor frente a la variación de la carga en el cable.

1.2.2. Objetivos específicos

- O1.** Desarrollar un modelo analítico del mecanismo que relacione la geometría, la tensión del cable y el par en el tambor. A partir de él, deducir el perfil de radio que hace el par

constante y compararlo con un tambor cilíndrico y con una espiral lineal.

- O2.** Diseñar una placa de circuito impreso que adapte el microcontrolador al entorno de la máquina: bus RS-485, entradas digitales protegidas para los finales de carrera, entradas analógicas acondicionadas, salidas de potencia y conexión de células de carga.
- O3.** Desarrollar el firmware del microcontrolador, capaz de gobernar el servo por Modbus RTU, leer la célula de carga, ejecutar el *homing* y los ciclos automáticos, y vigilar los límites de seguridad con una temporización determinista.
- O4.** Definir un protocolo de comunicación robusto entre el microcontrolador y el ordenador, que transporte la telemetría por lotes y las órdenes con detección de errores.
- O5.** Desarrollar una aplicación de supervisión web para operar la máquina, visualizar los datos en tiempo real, grabar cada ensayo en un fichero reproducible y compararlo con el modelo.
- O6.** Caracterizar el sistema (temporización, repetibilidad de posición, comportamiento de la medida) y analizar los ensayos para cuantificar el efecto compensador de la caracola.
- O7.** Documentar los problemas técnicos encontrados y las soluciones adoptadas, así como la planificación, el presupuesto y el impacto del trabajo.

1.3. Alcance y limitaciones

Entra en el alcance del trabajo todo el sistema electrónico y de software: la placa adaptadora, el firmware, el protocolo, la aplicación de supervisión y el análisis de datos, además del modelo del mecanismo y los ensayos. El diseño mecánico de la caracola y de la estructura se toma como dado **[COMPLETAR: indicar si la caracola y la estructura se han diseñado y fabricado dentro de este TFG o proceden de un trabajo previo]**.

El trabajo tiene las siguientes limitaciones conocidas, que se discuten en los capítulos correspondientes:

- La posición del eje no se lee del codificador del accionamiento, sino que se obtiene integrando la velocidad que devuelve el propio servo (sección 6.5).
- La célula de carga dispone sólo de una calibración provisional; sus valores en newtons se presentan como orientativos (sección 10.9).
- El par se obtiene en las unidades internas del accionamiento, cuya escala física no se ha confirmado. Por eso los resultados se expresan en variaciones relativas, que no dependen de esa escala.
- Los finales de carrera y el límite de fuerza se implementan por software; no hay una cadena de seguridad cableada (sección 4.5).

1.4. Metodología

El desarrollo ha sido incremental y guiado por pruebas. Cada subsistema (bus RS-485, célula de carga, finales de carrera, entradas y salidas de la placa) se ha validado primero con un programa de prueba aislado, que se conserva en el repositorio (anexo F), antes de integrarlo

en el firmware principal. El código se ha versionado con git desde el primer día; el historial de cambios ha servido también para reconstruir la planificación real del proyecto (capítulo 11). Los datos de todos los ensayos se guardan en crudo, con las constantes de conversión en la cabecera de cada fichero, de modo que cualquier resultado se puede recalcular sin repetir el ensayo. Las figuras de resultados de esta memoria se generan con un programa reproducible (`memoria/figuras/generar_figuras.py`) directamente a partir de esos ficheros.

1.5. Estructura del documento

Capítulo 2

repasa los mecanismos de radio variable y las tecnologías de accionamiento, medida de fuerza, comunicación y supervisión relevantes.

Capítulo 3

desarrolla el modelo del mecanismo, el perfil ideal de par constante y el método experimental para separar las componentes del par.

Capítulo 4

recoge los requisitos y la arquitectura del sistema.

Capítulos 5 a 8

describen, respectivamente, el hardware, el firmware, el protocolo de comunicación y la aplicación de supervisión.

Capítulo 9

reúne los problemas técnicos más relevantes y cómo se resolvieron.

Capítulo 10

presenta la caracterización del sistema y los resultados experimentales sobre la compensación de par.

Capítulo 11

trata la planificación, el presupuesto y el impacto.

Capítulo 12

expone las conclusiones y las líneas de trabajo futuro.

Los anexos contienen la asignación de pines, la especificación completa del protocolo, los registros Modbus utilizados, el formato de los ficheros de ensayo, un manual de uso y la estructura del repositorio.

Capítulo 2

Estado del arte y fundamentos

Este capítulo resume el contexto técnico del trabajo: primero los mecanismos de transmisión de radio variable, que son el objeto de estudio, y después las tecnologías con las que se ha construido el sistema de ensayo (accionamiento, medida de fuerza, microcontrolador, buses de comunicación y supervisión).

2.1. Mecanismos de transmisión con radio variable

2.1.1. Del *fusee* a los tambores de radio variable

El problema de convertir una fuerza variable en un par constante es antiguo. En los relojes de muelle, el par del muelle real disminuye a medida que se desenrolla, y eso altera la marcha. El *fusee* lo resolvió con un cono de perfil curvo y una ranura helicoidal por la que se enrolla una cadena: cuando el muelle está tenso, la cadena tira del radio pequeño del cono, y cuando se debilita, del radio grande, de manera que el par entregado al tren de engranajes se mantiene aproximadamente constante [1]. La condición de diseño es simplemente que el producto fuerza por radio se conserve a lo largo del recorrido.

La misma idea se ha formalizado en la robótica bajo el nombre de tambores de radio variable (*Variable Radius Drums*, VRD). Seriani y Gallina [3] plantean la síntesis del perfil de un tambor para que una carga siga una ley de fuerza prescrita, y obtienen soluciones analíticas cerradas para varios casos. En los robots paralelos accionados por cables, Bieber *et al.* [4] diseñan el perfil del tambor a partir del perfil de carga máxima en el espacio de trabajo, de manera que el par del motor no supere un valor dado para ninguna longitud de cable. La fabricación aditiva ha abaratado mucho la producción de estos perfiles, lo que explica el interés reciente.

2.1.2. Compensación de gravedad con poleas no circulares

Un segundo campo de aplicación es la compensación estática de peso. Herder [2] estudia de forma sistemática los mecanismos estáticamente equilibrados, en los que la energía potencial total se mantiene constante y el actuador sólo tiene que vencer las pérdidas. Endo *et al.* [6] proponen una polea no circular combinada con un muelle para compensar el par gravitatorio de un brazo, y deducen numéricamente la forma de la polea. Shen y Yano [5] refinan el cálculo de una polea en espiral teniendo en cuenta la variación de la tensión del cable, y mejoran así la calidad de la compensación.

El mecanismo de este trabajo comparte el principio con todos ellos, pero con una diferencia: aquí no hay un muelle que almacene energía, sino un motor que aporta todo el trabajo de

elevación. La caracola no reduce la energía necesaria para subir la masa; lo que hace es repartirla de forma uniforme a lo largo del giro, para que el par de pico sea el menor posible. El capítulo 3 demuestra que ese reparto uniforme es precisamente el óptimo.

2.2. Accionamiento: servomotores paso a paso de lazo cerrado

Un motor paso a paso convencional funciona en lazo abierto: el controlador impone la secuencia de fases y se da por hecho que el rotor la sigue. Es sencillo y tiene buen par a baja velocidad, pero si la carga supera el par disponible se pierden pasos sin que el sistema lo detecte [7]. Los servomotores paso a paso de lazo cerrado añaden un codificador al motor y un accionamiento que regula la corriente en función del error de posición. Conservan el par a baja velocidad del paso a paso, no pierden pasos y ofrecen varios modos de control (posición, velocidad o par), además de magnitudes de supervisión como la velocidad real o el par entregado.

El accionamiento usado en este trabajo pertenece a esta familia **[COMPLETAR: marca y modelo del motor y del accionamiento, par nominal, tensión de alimentación, resolución del codificador]**. Integra una interfaz RS-485 con protocolo Modbus RTU, a través de la cual se configura (modo de control, rigidez, rampas, resistencia de frenado), se habilita, se le envía la consigna de velocidad y se leen la velocidad y el par. Tener todo en un único bus de dos hilos simplifica mucho el cableado frente a la alternativa de señales de paso y dirección más entradas analógicas.

2.3. Medida de fuerza con células de carga

2.3.1. Galgas extensométricas y puente de Wheatstone

Una célula de carga convierte la fuerza en una deformación elástica, que se mide con galgas extensométricas dispuestas en un puente de Wheatstone [9]. La salida del puente es una tensión diferencial de unos pocos milivoltios por voltio de excitación a plena escala, que hay que amplificar con alta ganancia y bajo ruido antes de digitalizarla. Además hay que compensar el desequilibrio de cero del puente y su deriva con la temperatura [8].

2.3.2. Acondicionadores integrados

Existen circuitos integrados que agrupan en un solo chip la excitación, la amplificación, la conversión A/D y la corrección digital. En este trabajo se han evaluado dos:

- **ZSC31014** de Renesas [21]. Acondicionador con salida digital I²C o SPI, preamplificador de ganancia programable, conversor de 14 bits y corrección de ganancia y cero almacenada en EEPROM. Cada lectura incluye dos bits de estado que indican si el dato es nuevo o ya se había leído. Su configuración sólo puede modificarse en un *modo comando* al que se accede durante una ventana corta tras el encendido, lo que ha condicionado mucho el diseño (sección 9.2). Se ha usado integrado en la placa *Load Cell 2 Click* de MikroElektronika [27].
- **NAU7802** de Nuvoton [22]. Conversor sigma-delta de 24 bits con amplificador programable de hasta 128 y regulador interno. Se probó como segunda célula (programa

`test_loadcell_dual_v2`) y se retiró del firmware final, que trabaja con una sola célula.

2.4. Microcontrolador

El proyecto empezó con una placa basada en ESP32-S3 (Seeed XIAO) y se migró a una Raspberry Pi Pico 2. La tabla 2.1 resume las características relevantes del RP2350 [20]. La razón principal del cambio, y sus consecuencias, se explican en la sección 9.1: el RP2350 no tiene UART hardware en los pines que usa la placa, pero sí periféricos PIO que pueden implementar una UART en cualquier pin.

Tabla 2.1: Características del microcontrolador RP2350 en la Raspberry Pi Pico 2 [20].

Característica	Raspberry Pi Pico 2 (RP2350)
Núcleos	$2 \times$ Arm Cortex-M33 o $2 \times$ RISC-V Hazard3, a 150 MHz
Memoria	520 kB SRAM, 4 MB flash QSPI
Periféricos serie	$2 \times$ UART, $2 \times$ SPI, $2 \times$ I ² C
E/S programable	12 máquinas de estado PIO
Entradas analógicas	3 canales ADC
Entorno usado	Arduino-Pico [28]

2.5. Buses de comunicación

2.5.1. RS-485 y Modbus RTU

RS-485 [12] es un estándar de capa física diferencial y multipunto, muy extendido en la industria por su inmunidad al ruido, su alcance y la posibilidad de conectar varios equipos a un mismo par trenzado. En modo semidúplex, cada nodo activa su emisor sólo mientras transmite; el control de esa dirección (líneas DE/RE del transceptor) es responsabilidad del maestro.

Modbus [10] es un protocolo maestro-esclavo de petición y respuesta que modela los datos del esclavo como registros de 16 bits. En su variante RTU sobre línea serie [11], cada trama lleva la dirección del esclavo, el código de función, los datos y un CRC-16. Las tramas se delimitan por silencios en la línea de al menos 3,5 caracteres. En este trabajo se usan la función 0x03 (lectura de registros) y la 0x06 (escritura de un registro), a través de la biblioteca ModbusMaster [29].

2.5.2. I²C

I²C [13] es un bus síncrono de dos hilos con resistencias de *pull-up*, pensado para comunicar circuitos dentro de una misma placa. Es el bus que usan los acondicionadores de célula de carga. Las placas Click siguen el estándar mikroBUS [14], que define un zócalo de 16 pines con I²C, SPI, UART, alimentación y señales auxiliares.

2.6. Supervisión y adquisición en el ordenador

La supervisión de máquinas se ha hecho tradicionalmente con paneles HMI o paquetes SCADA comerciales. Para un banco de ensayo de laboratorio, una aplicación web local es una alternativa ligera: no requiere licencias, funciona en cualquier navegador y permite a varios clientes ver la máquina a la vez. En este trabajo se ha usado FastAPI [30] sobre el servidor ASGI Uvicorn [31], pySerial [32] para el puerto USB, WebSocket [15] para el envío continuo de datos al navegador y Chart.js [33] para las gráficas.

2.7. Conclusiones del estado del arte

Los tambores de radio variable están bien estudiados como principio, y en la literatura se encuentran métodos de síntesis de su perfil. Lo que motiva este trabajo es disponer de una plataforma de bajo coste, construida con componentes comerciales, que permita medir en un prototipo real cuánto aplanan el par una caracola y comparar la medida con el modelo. Para ello hay que integrar un accionamiento industrial por Modbus, un acondicionador de célula de carga con reglas de configuración estrictas y una cadena de adquisición con marcas de tiempo fiables, que es lo que desarrollan los capítulos siguientes.

Capítulo 3

Modelo del mecanismo

Este capítulo desarrolla el modelo estático del mecanismo. Relaciona el ángulo de la barra con la longitud del cable y la tensión, y a partir de ahí obtiene el par en el tambor para tres perfiles: cilíndrico, espiral lineal y perfil ideal de par constante. Termina con el método que se usará para separar, en el par medido, la parte debida a la gravedad de la debida a las pérdidas. Las mismas ecuaciones están implementadas en las páginas de supervisión ([static/index.html](#) y [static/ensayos.html](#)), lo que permite superponer la curva teórica a cada ensayo.

3.1. Descripción del mecanismo

La figura 3.1 muestra el esquema. Una barra rígida está articulada en el punto O y lleva suspendida una masa m a una distancia $L_3 + L_4$ de la articulación. Un cable se ancla a la barra en el punto P, a una distancia L_3 de O, y sube hasta el punto de salida E, situado a una altura L_2 sobre la vertical de O. Desde E, el cable se enrolla en la caracola, que gira solidaria al eje de salida de la transmisión del servo, con una reducción N motor:tambor. El ángulo θ de la barra se mide desde la horizontal, positivo hacia arriba; su recorrido físico va de $+90^\circ$ (masa arriba) a -90° (masa abajo).

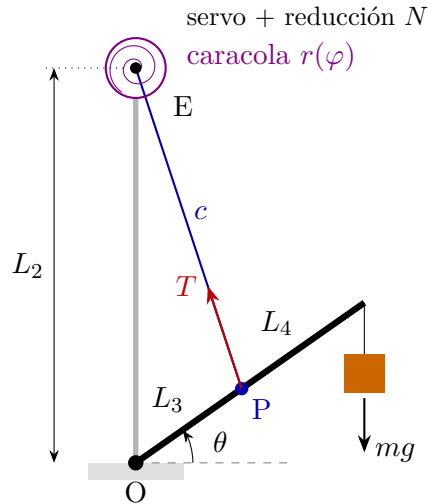


Figura 3.1: Esquema del mecanismo de elevación con caracola.

El modelo parte de las siguientes hipótesis:

H1. Régimen cuasiestático: a velocidad constante se desprecian las fuerzas de inercia.

- H2. Cable inextensible y sin masa; el punto de salida E es fijo (polea o tambor de dimensión despreciable frente a L_2).
- H3. Se desprecia la masa de la barra frente a la masa suspendida **[COMPLETAR: si la masa de la barra no es despreciable, añadir su momento $m_b g x_G \cos \theta$ en la ecuación (3.4); la estructura del resultado no cambia]**.
- H4. Las pérdidas por rozamiento no entran en el modelo estático; se tratan aparte en la sección 3.5.

3.2. Longitud del cable

Con O en el origen, E en $(0, L_2)$ y P en $(L_3 \cos \theta, L_3 \sin \theta)$, la longitud del cable es la distancia entre E y P:

$$c(\theta)^2 = L_2^2 + L_3^2 - 2L_2L_3 \sin \theta. \quad (3.1)$$

La función es monótona decreciente en $\theta \in [-90^\circ, 90^\circ]$, así que cada longitud de cable corresponde a un único ángulo. Los valores extremos son $c_{\min} = |L_2 - L_3|$ con la barra vertical hacia arriba y $c_{\max} = L_2 + L_3$ con la barra vertical hacia abajo. Despejando,

$$\sin \theta = \frac{L_2^2 + L_3^2 - c^2}{2L_2L_3}. \quad (3.2)$$

3.3. Tensión del cable

El momento del peso respecto a O es $mg(L_3 + L_4) \cos \theta$. El brazo de la tensión respecto a O se obtiene del producto vectorial $\overrightarrow{OP} \times \hat{u}_{PE}$, con $\overrightarrow{PE} = (-L_3 \cos \theta, L_2 - L_3 \sin \theta)$:

$$d = \frac{|L_3 \cos \theta (L_2 - L_3 \sin \theta) + L_3 \sin \theta L_3 \cos \theta|}{c} = \frac{L_2L_3 \cos \theta}{c}. \quad (3.3)$$

Igualando momentos,

$$T \frac{L_2L_3 \cos \theta}{c} = mg(L_3 + L_4) \cos \theta, \quad (3.4)$$

el factor $\cos \theta$ se cancela y queda un resultado notablemente simple:

$$\boxed{T = K c, \quad K = \frac{mg(L_3 + L_4)}{L_2L_3}} \quad (3.5)$$

La tensión es proporcional a la longitud libre de cable. En las posiciones verticales ($\theta = \pm 90^\circ$) los dos brazos se anulan y el equilibrio queda indeterminado; la expresión (3.5) da el límite por continuidad.

Con la geometría nominal de la tabla 3.1, la tensión varía entre 28,5 N con la masa arriba y 56,2 N con la masa abajo, es decir, se duplica a lo largo del recorrido completo (figura 3.2, izquierda).

Tabla 3.1: Geometría nominal empleada en el modelo. Son los valores por defecto de la interfaz de supervisión [VERIFICAR: sustituir por las cotas medidas en el prototipo y regenerar las figuras].

Parámetro	Valor	Comentario
L_2	290 mm	altura de la salida del cable
L_3	95 mm	anclaje del cable en la barra
L_4	110 mm	resto de barra hasta la masa
m	2 kg	masa suspendida
K	0,1460 N/mm	ecuación (3.5)
$c_{\min} - c_{\max}$	195 mm a 385 mm	recorrido de cable $\Delta c = 190$ mm
φ_{tot}	0,5 rev de tambor	barrido de la caracola
N	4	reducción motor:tambor

3.4. Par en el tambor

Si el tambor tiene un radio de enrollamiento $r(\varphi)$ que depende de su giro φ , el cable pagado es $s(\varphi) = \int_0^\varphi r(u) du$, y la longitud libre, tomando como origen la masa arriba y el sentido en que el cable se desenrolla, es

$$c(\varphi) = c_{\min} + s(\varphi), \quad \frac{dc}{d\varphi} = r(\varphi). \quad (3.6)$$

El par en el eje del tambor y en el eje del motor son

$$\tau(\varphi) = T r(\varphi) = K c(\varphi) r(\varphi), \quad \tau_m = \frac{\tau}{N \eta}, \quad (3.7)$$

donde η es el rendimiento de la transmisión, y el giro del tambor se relaciona con la posición del motor p (en revoluciones) mediante $\varphi = 2\pi p/N$.

3.4.1. Cota inferior del par de pico

Cualquiera que sea el perfil, el trabajo realizado sobre el tambor a lo largo del recorrido completo es igual al aumento de energía potencial de la masa:

$$W = \int_0^{\varphi_{\text{tot}}} \tau d\varphi = \int_{c_{\min}}^{c_{\max}} T dc = \frac{K}{2} (c_{\max}^2 - c_{\min}^2) = mg (L_3 + L_4) (\sin \theta_1 - \sin \theta_2). \quad (3.8)$$

Para el recorrido completo (θ de $+90^\circ$ a -90°), $W = 2mg(L_3 + L_4) = 8,044$ J. Como la integral del par sobre $[0, \varphi_{\text{tot}}]$ vale W , su valor máximo no puede ser menor que su valor medio:

$$\max_{\varphi} \tau(\varphi) \geq \frac{W}{\varphi_{\text{tot}}}. \quad (3.9)$$

La igualdad sólo se alcanza si el par es constante. *El perfil de par constante es, por tanto, el que minimiza el par de pico para un recorrido y un giro de tambor dados.*

3.4.2. Tambor cilíndrico

Con $r = r_c$ constante, el cable se paga de forma lineal. Para cubrir el recorrido en el mismo giro hace falta $r_c = \Delta c / \varphi_{\text{tot}} = 60,5 \text{ mm}$, y el par $\tau = K c r_c$ crece linealmente con el giro, de 1,72 a 3,40 N m. El cociente entre el par de pico del perfil ideal y el del cilindro tiene una expresión cerrada:

$$\frac{\tau_{\text{ideal}}}{\tau_{\text{cil,max}}} = \frac{K (c_{\text{máx}}^2 - c_{\text{mín}}^2) / (2\varphi_{\text{tot}})}{K c_{\text{máx}} \Delta c / \varphi_{\text{tot}}} = \frac{c_{\text{máx}} + c_{\text{mín}}}{2 c_{\text{máx}}}, \quad (3.10)$$

que sólo depende de la geometría. Con los valores nominales vale $580/770 = 0,753$: la caracola ideal reduce el par de pico un 24,7 %.

3.4.3. Perfil ideal de par constante

Imponiendo $\tau = K c r = \text{cte}$ y usando (3.6), $c \, dc/d\varphi = C$ con $C = \tau/K$. Integrando desde $c(0) = c_0$:

$$\boxed{c(\varphi) = \sqrt{c_0^2 + 2C\varphi}, \quad r(\varphi) = \frac{C}{\sqrt{c_0^2 + 2C\varphi}}} \quad \text{con} \quad C = \frac{c_{\text{máx}}^2 - c_{\text{mín}}^2}{2\varphi_{\text{tot}}}. \quad (3.11)$$

El radio ideal es inversamente proporcional a la longitud de cable, igual que el *fusee* compensa un muelle cuya fuerza decrece. Con la geometría nominal, $C = 17.539 \text{ mm}^2/\text{rad}$, el radio pasa de 89,9 mm a 45,6 mm (una relación de 2 a 1) y el par es constante e igual a $W/\varphi_{\text{tot}} = 2,56 \text{ N m}$ en el tambor, o 0,64 N m en el motor con $N = 4$ y $\eta = 1$.

La consecuencia práctica es que el radio debe ser *grande donde la tensión es pequeña* (masa arriba) y *pequeño donde la tensión es grande* (masa abajo).

3.4.4. Espiral de radio lineal

Una espiral de Arquímedes, con radio lineal en el giro, es más sencilla de fabricar y es el perfil que modela la página de ensayos:

$$r(\varphi) = r_0 + (r_1 - r_0) \frac{\varphi}{\varphi_{\text{tot}}}, \quad s(\varphi) = r_0 \varphi + \frac{(r_1 - r_0) \varphi^2}{2\varphi_{\text{tot}}}. \quad (3.12)$$

Para compararla en igualdad de condiciones, se ha buscado numéricamente la relación r_1/r_0 que minimiza el par de pico con la restricción de pagar exactamente Δc en φ_{tot} . El óptimo es $r_0 = 77,2 \text{ mm}$, $r_1 = 43,7 \text{ mm}$, con un par entre 2,20 y 2,68 N m.

3.4.5. Comparación

La figura 3.2 y la tabla 3.2 resumen los tres perfiles. Los tres realizan el mismo trabajo (8,044 J, comprobado numéricamente), pero lo reparten de forma muy distinta. La espiral lineal consigue el 85 % de la reducción de pico alcanzable y baja el rizado de par del 66 % al 19 %.

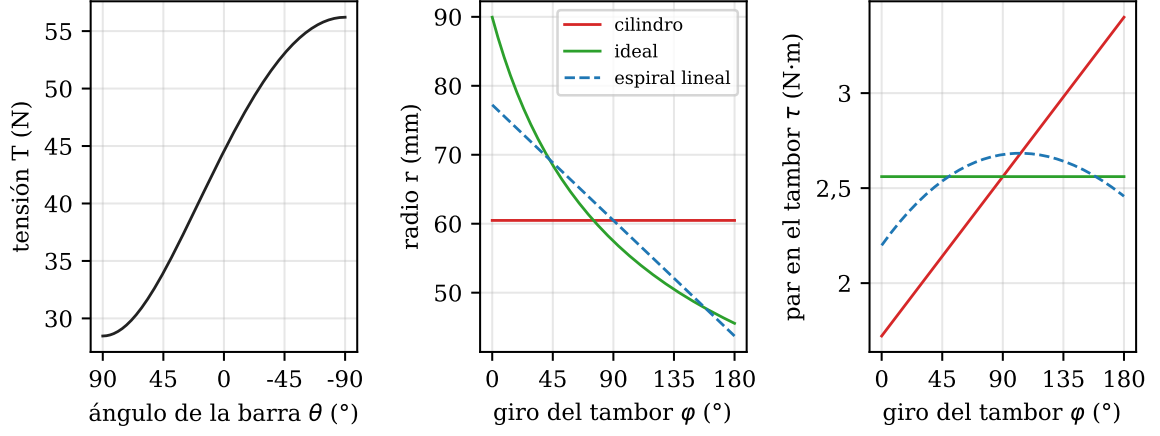


Figura 3.2: Modelo con la geometría nominal. Izquierda: tensión del cable frente al ángulo de la barra. Centro: radio de enrollamiento de los tres perfiles. Derecha: par en el tambor.

Tabla 3.2: Comparación de perfiles de tambor con la geometría nominal (recorrido completo, $\varphi_{\text{tot}} = 0,5 \text{ rev}$).

Perfil	r inicial (mm)	r final (mm)	τ mín. (N · m)	τ máx. (N · m)	Rizado $\frac{\text{máx} - \text{mín}}{\text{media}}$
Cilindro	60,5	60,5	1,72	3,40	66 %
Espiral lineal	77,2	43,7	2,20	2,68	19 %
Ideal	89,9	45,6	2,56	2,56	0 %

Reducción del par de pico frente al cilindro: lineal 21,1 %; ideal 24,7 %.

3.5. Separación experimental de las componentes del par

El accionamiento entrega el par en unidades internas cuya escala no se ha confirmado, y el par medido incluye además las pérdidas por rozamiento de la transmisión y del cable. Para comparar la medida con el modelo sin depender de esa escala se propone el siguiente método.

Si se recorre el mismo punto p subiendo la masa (sentido A en el prototipo) y bajándola (sentido B) a la misma velocidad constante, las pérdidas se oponen al movimiento en ambos casos, mientras que el par gravitatorio conserva el signo. Suponiendo pérdidas simétricas,

$$|\tau_A(p)| = \tau_g(p) + \tau_f(p), \quad |\tau_B(p)| = \tau_g(p) - \tau_f(p), \quad (3.13)$$

de donde

$$\tau_g = \frac{|\tau_A| + |\tau_B|}{2}, \quad \tau_f = \frac{|\tau_A| - |\tau_B|}{2}. \quad (3.14)$$

El mismo promedio aplicado a la fuerza medida en ambos sentidos elimina en primera aproximación el retardo del filtro de la célula de carga, que desplaza la curva de subida y la de bajada en sentidos opuestos. Por último, la ecuación (3.7) permite estimar un *radio efectivo relativo*

$$r_{\text{ef}}(p) \propto \frac{\tau_g(p)}{T(p)}, \quad (3.15)$$

cuya forma no depende de la escala del par ni de la reducción. En un tambor cilíndrico sería constante, y en una caracola compensadora debe decrecer a medida que aumenta la tensión.

3.6. Limitaciones del modelo

- El punto de salida del cable del tambor real se desplaza ligeramente con el radio, lo que modifica L_2 en unos milímetros.
- Se desprecian la elasticidad del cable y la masa de la barra.
- El rendimiento de la transmisión y las pérdidas se suponen independientes del sentido de giro, en valor absoluto, para aplicar (3.14).
- Las cotas de la tabla 3.1 son nominales; con las cotas reales cambian los valores numéricos, pero no las conclusiones cualitativas ni las expresiones (3.9)–(3.11).

Capítulo 4

Requisitos y arquitectura del sistema

4.1. Requisitos

Los requisitos se derivan de lo que exige el ensayo de la caracola (capítulo 3). El eje debe moverse a velocidad constante y conocida en ambos sentidos, y hay que registrar simultáneamente, con la misma base de tiempos, la posición, la tensión del cable y el par del motor. Todo ello debe hacerse de forma segura y repetible.

4.1.1. Requisitos funcionales

Id.	Requisito
RF1	Configurar y habilitar el accionamiento y gobernar su velocidad a través de Modbus RTU sobre RS-485.
RF2	Leer la velocidad y el par del motor en cada periodo de control.
RF3	Leer la tensión del cable con una célula de carga y un acondicionador digital.
RF4	Detectar los finales de carrera A y B y detener el movimiento hacia un final activado.
RF5	Ejecutar una rutina de <i>homing</i> que localice A, mida el recorrido hasta B y fije el origen en un desplazamiento configurable.
RF6	Ejecutar ciclos automáticos $A \leftrightarrow B$, con B en el final de carrera o en una posición dada, a una velocidad y un número de repeticiones configurables.
RF7	Enviar al ordenador la telemetría de cada periodo con su marca de tiempo del dispositivo.
RF8	Recibir órdenes y parámetros desde el ordenador con detección de errores.
RF9	Consultar y programar la ganancia y el cero del acondicionador de la célula, sólo con el eje parado.
RF10	Mostrar el estado de la máquina y los datos en tiempo real en un navegador.
RF11	Grabar automáticamente cada ciclo en un fichero con datos brutos y constantes de conversión.
RF12	Visualizar los ensayos grabados y compararlos con el modelo teórico.

4.1.2. Requisitos no funcionales

Id.	Requisito
RNF1	Determinismo: periodo de control y muestreo fijo de 50 ms, con dispersión inferior a 1 ms.
RNF2	Latencia de parada: una orden de parada, un final de carrera o el límite de fuerza deben anular la consigna en el mismo periodo en que se detectan.
RNF3	Robustez de la comunicación: una trama corrupta no debe ejecutar ninguna orden, y el receptor debe resincronizarse solo.
RNF4	Recuperación: la aplicación debe reconectarse sola si se desconecta o reinicia el microcontrolador.
RNF5	Reproducibilidad: cualquier resultado debe poder recalcularse a partir de los ficheros grabados.
RNF6	Coste y disponibilidad: componentes comerciales y herramientas libres.
RNF7	Mantenibilidad: validación de cada subsistema con un programa de prueba propio y código documentado.

4.2. Arquitectura general

La figura 4.1 muestra la arquitectura. El sistema se organiza en tres niveles:

1. **Nivel de campo:** el servomotor paso a paso con su accionamiento, la transmisión, la caracola, el cable y la barra con la masa; la célula de carga intercalada en el cable y los dos finales de carrera.
2. **Nivel de control:** la placa adaptadora con la Raspberry Pi Pico 2. Ejecuta el lazo de 50 ms y es el único elemento que actúa sobre la máquina.
3. **Nivel de supervisión:** un ordenador con el servidor `web_monitor.py`, que traduce entre el protocolo binario del puerto USB y mensajes JSON por WebSocket, graba los ensayos y sirve las páginas de supervisión y de análisis.

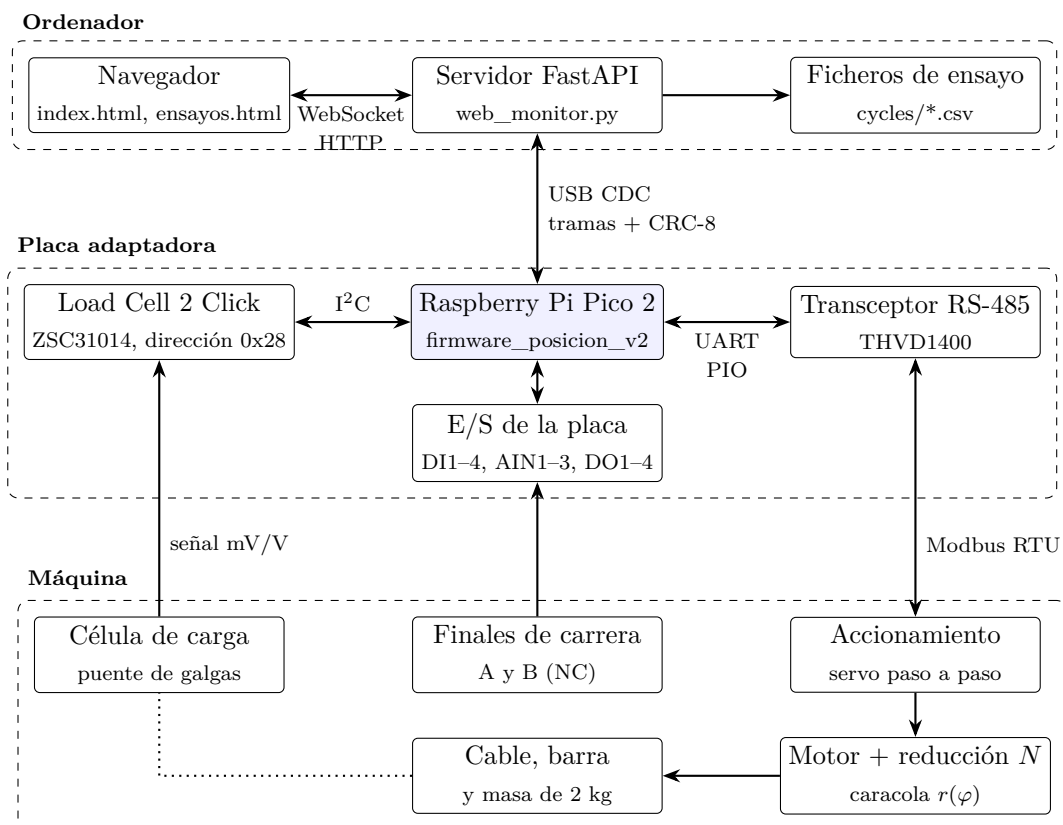


Figura 4.1: Arquitectura del sistema de accionamiento, control y adquisición.

4.3. Flujo de datos y temporización

El microcontrolador ejecuta un lazo de periodo fijo, que se describe en detalle en la sección 6.2. En cada vuelta lee el estado del servo y de la célula, integra la posición, procesa las órdenes recibidas, actualiza las máquinas de estado de *homing* y de ciclo, aplica las protecciones y escribe la nueva consigna. Además guarda una muestra de la vuelta en un búfer. Cada 200 ms (o antes, si el búfer se llena) el búfer se envía en una sola trama, con una marca de tiempo por muestra. El servidor desempaqueta las muestras, las reenvía al navegador y, si hay un ciclo en marcha, las escribe en el fichero del ensayo. Toda la cadena usa el reloj del microcontrolador, de modo que las gráficas y los ficheros reflejan los instantes reales de medida y no los de llegada al ordenador.

4.4. Decisiones de diseño

La tabla 4.3 resume las decisiones de arquitectura más importantes. Varias de ellas son consecuencia de problemas encontrados durante el desarrollo, que se analizan en el capítulo 9.

Tabla 4.3: Decisiones de diseño y alternativas consideradas.

Aspecto	Elección	Justificación / alternativa descartada
Microcontrolador	Raspberry Pi Pico 2	Sustituye al ESP32-S3 inicial. La placa adaptadora se diseñó para el módulo Pico, y su PIO permite una UART en los pines que imponía el trazado.
Modo del servo	Velocidad	Los primeros firmwares usaban el modo par con un regulador proporcional sobre la célula de carga. Para el ensayo cuasiestático y para el método de separación del par (sección 3.5) hace falta velocidad constante e igual en ambos sentidos, así que se pasó al modo velocidad.
Organización del firmware	Un único dueño de cada bus en el núcleo 0	La versión de doble núcleo dedicaba el núcleo 1 a Modbus con variables compartidas. La versión final ejecuta todo en un lazo con periodo fijo, lo que elimina las condiciones de carrera y hace la temporización predecible.
Protocolo con el PC	Binario con CRC-8 y telemetría por lotes	Un protocolo de texto ocupa más y es ambiguo de delimitar. Las tramas binarias con cabecera, longitud y CRC se resincronizan solas y rechazan la corrupción.
Posición	Integración de la velocidad del servo	No requiere cablear el codificador. Su deriva se acota con el <i>homing</i> y los finales de carrera (sección 6.5).
Células de carga	Una sola (ZSC31014)	El NAU7802 se evaluó como segunda célula y se retiró. Compartían bus, y el ZSC31014 no tolera transacciones ajenas entre la petición y la recogida del dato.
Supervisión	Aplicación web local	No requiere licencias, admite varios clientes y separa la operación en vivo del análisis de ensayos.
Datos grabados	Cuentas brutas + constantes en la cabecera	Permite recalibrar un ensayo sin repetirlo.

4.5. Seguridad

El prototipo mueve una masa pequeña a baja velocidad, pero el sistema incluye varias medidas de seguridad, todas implementadas en el firmware:

- **Finales de carrera normalmente cerrados** conectados entre la entrada y masa, con *pull-up* interna. En reposo la entrada lee nivel bajo, y un final pulsado o un cable

roto dan nivel alto. La rotura de cable se interpreta, por tanto, como final activado (comportamiento a prueba de fallos).

- **Parada hacia el final activado:** en cada periodo se anula cualquier consigna que empuje hacia un final pulsado, independientemente del modo (manual, *homing* o ciclo).
- **Límite de fuerza** configurable: si el valor absoluto de la célula tarada supera el umbral, la consigna se anula. Sólo actúa con una lectura válida reciente (menos de 150 ms).
- **Temporizadores de tramo** de 30 s en el *homing* y en los ciclos: un tramo que no termina se aborta.
- **Bloqueo durante la programación de la célula:** grabar la EEPROM deja la célula sin medida durante cientos de milisegundos, así que se rechaza si el eje no está parado y, mientras dura, se fuerza la consigna a cero en cada vuelta.
- **Orden de parada** que corta también el *homing* y los ciclos, y orden de apagado que deshabilita el accionamiento.

Estas medidas dependen del software y del enlace Modbus. En una máquina industrial, las normas de seguridad funcional exigirían una cadena de parada independiente del control [16], por ejemplo cableando los finales de carrera a la entrada de desconexión segura del par (STO) del accionamiento [17]. Se recoge como trabajo futuro en el capítulo 12.

Capítulo 5

Diseño hardware

5.1. Placa adaptadora: objetivos

El módulo Raspberry Pi Pico 2 trabaja a 3,3 V y no puede conectarse directamente al entorno de la máquina. La placa adaptadora (proyecto KiCad `tfg_adaptador`) resuelve esa adaptación y reúne en un solo circuito:

- un transceptor RS-485 semidúplex para el bus Modbus del accionamiento;
- cuatro entradas digitales protegidas para finales de carrera y otros contactos;
- entradas analógicas de 0 V a 5 V acondicionadas para el ADC del microcontrolador;
- cuatro salidas de potencia de lado alto con diagnóstico;
- dos zócalos mikroBUS para placas Click de célula de carga;
- bornas de tornillo para todo el cableado de campo y taladros de fijación M3.

5.2. Revisiones del diseño

El repositorio contiene dos estados del diseño, que conviene distinguir:

- **Revisión A:** los ficheros `tfg_adaptador.kicad_sch` y `.kicad_pcb`, creados entre el 12 y el 17 de marzo de 2026. Tienen el esquema, la colocación de los componentes y los planos de cobre (figuras 5.2 y 5.3).
- **Revisión B:** el esquema de la figura 5.1, incorporado al repositorio como imagen. Es la asignación de pines que usan todos los programas de prueba de la placa y el firmware final [**VERIFICAR:** añadir al repositorio los ficheros KiCad de la revisión B, que no están versionados, y regenerar las figuras de la PCB].

El análisis de la lista de conexiones de la revisión A, extraída con `kicad-cli`, muestra una serie de problemas que la revisión B corrige (tabla 5.1). Documentarlos es útil, porque son errores típicos de un primer diseño.

Tabla 5.1: Problemas detectados en la revisión A y corrección en la revisión B.

Bloque	Revisión A (lista de conexiones)	Revisión B
RS-485	La salida RO del transceptor está en la misma red que 3,3 V y GPIO13. DI, DE y $\overline{RE}$ están unidas entre sí y a GPIO14 y GPIO15. El transceptor no puede funcionar en semidúplex.	DE y $\overline{RE}$ a GPIO12 con <i>pull-down</i> de 10 k Ω ; RO a GPIO13 con <i>pull-up</i> de 10 k Ω ; DI a GPIO14.
Entradas analógicas	Las salidas de los seguidores llegan a GPIO16–19, que no tienen ADC.	Salidas a GPIO26–28 (ADC0–2).
Amplificadores	Cuatro MCP6001 sin alimentación: pines V+ y V– sin conectar.	Un MCP6294 cuádruple alimentado a 5 V con desacoplo de 100 nF.
Salidas de potencia	El TPS4H000 se alimenta de VBUS (5 V del USB); diagnóstico ($\overline{ST}$), DIAG_EN, CL y THER sin conectar; entradas directas.	Alimentación desde la borna VS_IN; $\overline{ST1}$ – $\overline{ST4}$ unidas a DST con <i>pull-up</i> ; DIAG_EN controlado; resistencias de 10 k Ω en serie con las entradas.
Células de carga	Cada zócalo en un bus I ² C distinto (GPIO0/1 y GPIO2/3), sin control de EN ni lectura de INT.	Ambos zócalos en I ² C1 (GPIO2/3), habilitación EN1 (GPIO1) e interrupciones INT1 (GPIO0) e INT2 (GPIO4).
Entradas digitales	Llegan a GPIO4–7.	Llegan a GPIO16–19.
Rutado	El fichero de PCB no contiene pistas (0 segmentos): sólo colocación, 23 vías y planos.	[COMPLETAR: indicar cómo se fabricó la placa definitiva: PCB de revisión B, placa perforada o cableado manual]

5.3. Microcontrolador

La placa aloja el módulo Raspberry Pi Pico 2 en sus dos filas de 20 pines, de modo que puede sustituirse sin desoldar. El módulo se alimenta y comunica por USB. De él se toman 5 V (VBUS) para los amplificadores y 3,3 V (salida 3V3 del regulador del módulo) para el transceptor y las placas Click. La asignación completa de pines está en el anexo A.

5.4. Interfaz RS-485

Se usa el transceptor THVD1400 de Texas Instruments [23]: semidúplex, alimentación de 3 V a 5,5 V, hasta 500 kbit/s, 1/8 de carga unitaria (hasta 256 nodos), protección ESD de

± 16 kV HBM y ± 12 kV IEC 61000-4-2 por contacto, y protección ante fallos de bus de ± 16 V. Esta robustez es adecuada para un cable que convive con los de potencia del motor.

Las señales DE y $\overline{\text{RE}}$ están unidas y se controlan desde GPIO12. Una resistencia de 10 k Ω a masa deja el transceptor en recepción durante el arranque, cuando el pin aún no está configurado, para que no ocupe el bus. La salida RO lleva una *pull-up* de 10 k Ω para que la línea de recepción no quede flotando. El bus sale por la borna de tres polos J5 (A, B y masa). El firmware activa el emisor antes de cada trama y lo desactiva 200 μ s después del último byte (sección 9.1). No hay terminación en la placa: el enlace es punto a punto y corto [VERIFICAR: longitud del cable y si el accionamiento incorpora terminación de 120 Ω].

5.5. Entradas digitales

Cada entrada digital (DI1–DI4) pasa por una resistencia serie de 1 k Ω y un diodo zéner BZX384 a masa [26] antes de llegar al GPIO (figura 5.1). La resistencia limita la corriente y el zéner recorta las sobretensiones y las descargas que entran por el cable. Los finales de carrera, de contacto normalmente cerrado, se conectan entre la borna y masa, y el firmware activa la *pull-up* interna del GPIO.

La tensión que admite la entrada de forma continua está limitada por la potencia de la resistencia. Con un zéner de tensión V_Z y una resistencia de 0,125 W (encapsulado 0805), la tensión de entrada máxima es

$$V_{\text{in,max}} = V_Z + \sqrt{P_R R} \approx V_Z + \sqrt{0,125 \cdot 1000} \approx V_Z + 11,2 \text{ V}. \quad (5.1)$$

Para contactos secos es más que suficiente. Si se quisieran conectar sensores de 24 V, habría que subir la resistencia a unos 10 k Ω [COMPLETAR: tensión nominal del zéner BZX384 montado].

5.6. Entradas analógicas

Las entradas AIN1–AIN3 admiten 0 V a 5 V. Cada canal tiene un divisor resistivo de 17 k Ω y 33 k Ω , con relación $33/50 = 0,66$, que lleva 5 V a 3,3 V, seguido de un amplificador operacional en seguidor de tensión. El seguidor presenta al ADC una impedancia de fuente baja, necesaria para cargar el condensador de muestreo en cada conversión sin error, y aísla el divisor del ADC. Se usa el MCP6294 [25], cuádruple, *rail-to-rail* en entrada y salida y alimentado a 5 V.

Con el ADC de 12 bits del RP2350 y referencia de 3,3 V, la resolución referida a la entrada es

$$\Delta V_{\text{in}} = \frac{3,3 \text{ V}}{4095 \cdot 0,66} \approx 1,22 \text{ mV}. \quad (5.2)$$

El cuarto amplificador se usa como comparador con un umbral de 2,5 V, obtenido de 5 V con un divisor de dos resistencias de 10 k Ω . Su salida se adapta a 3,3 V con otro divisor 17/33 y llega a una entrada digital (COMP).

El valor de 17 k Ω no pertenece a la serie E24. Para la fabricación conviene usar 17,4 k Ω (E96), con relación 0,655, o recalibrar el factor en el firmware.

5.7. Salidas de potencia

Las cuatro salidas usan el interruptor inteligente de lado alto TPS4H000-Q1 [24]: cuatro canales, alimentación de 3,4 V a 40 V, 1000 m Ω de resistencia en conducción, límite de corriente ajustable entre 0,1 y 1,3 A, detección de sobrecorriente, cortocircuito y carga abierta, protección térmica y capacidad para cargas inductivas. Las entradas IN1–IN4 se gobiernan desde GPIO7–10 a través de resistencias de 10 k Ω . Las salidas de estado, en drenador abierto, se unen en una línea común de fallo (DST) con *pull-up*, y la habilitación del diagnóstico se controla con DIAG_EN. El firmware final no usa todavía estas salidas; su funcionamiento se verificó con el programa `pcbtest/test_digital_outputs` **[COMPLETAR: uso previsto de las salidas y resistencia de límite de corriente elegida]**.

5.8. Célula de carga

La célula de carga se conecta a una placa *Load Cell 2 Click* de MikroElektronika [27], basada en el acondicionador ZSC31014 [21], que se inserta en el zócalo mikroBUS MD1. El chip responde en la dirección I²C 0x28, admite alimentación de 2,7 V a 5,5 V y ofrece ocho ganancias de preamplificador ($\times 1,5$ a $\times 192$). Su señal EN, controlada desde GPIO1 con la corriente de salida del pin configurada a 12 mA, permite apagarlo y encenderlo para forzar un reinicio y abrir la ventana del modo comando. Esta posibilidad resultó imprescindible, y también problemática (sección 9.2).

El segundo zócalo (MD2) comparte el bus y se usó para evaluar una *Load Cell 4 Click* con NAU7802 [22] **[COMPLETAR: modelo, capacidad nominal y sensibilidad (mV/V) de la célula de carga, y forma de montaje en el cable]**.

5.9. Alimentación

La lógica se alimenta del USB del ordenador a través del módulo Pico. Las salidas de potencia toman su tensión de la borna VS_IN y el accionamiento tiene su propia fuente **[COMPLETAR: tensión y potencia de la fuente del accionamiento]**. Las masas se unen en la placa, y la masa del bus RS-485 se lleva también a la borna J5 para dar al transceptor una referencia común con el accionamiento.

5.10. Placa de circuito impreso

La PCB de la revisión A es de dos capas, mide 123 mm $\times$ 75 mm y tiene cuatro taladros M3 en las esquinas. Los componentes pasivos son SMD 0805, los diodos SOD-323, los amplificadores SOT-23-5, el transceptor SOIC-8 y el interruptor de potencia HTSSOP-20 con almohadilla térmica. Las bornas de campo están agrupadas en el lado izquierdo, el módulo Pico en el lado derecho con el conector USB hacia el borde, y los dos zócalos mikroBUS arriba a la derecha, de modo que las placas Click quedan accesibles.

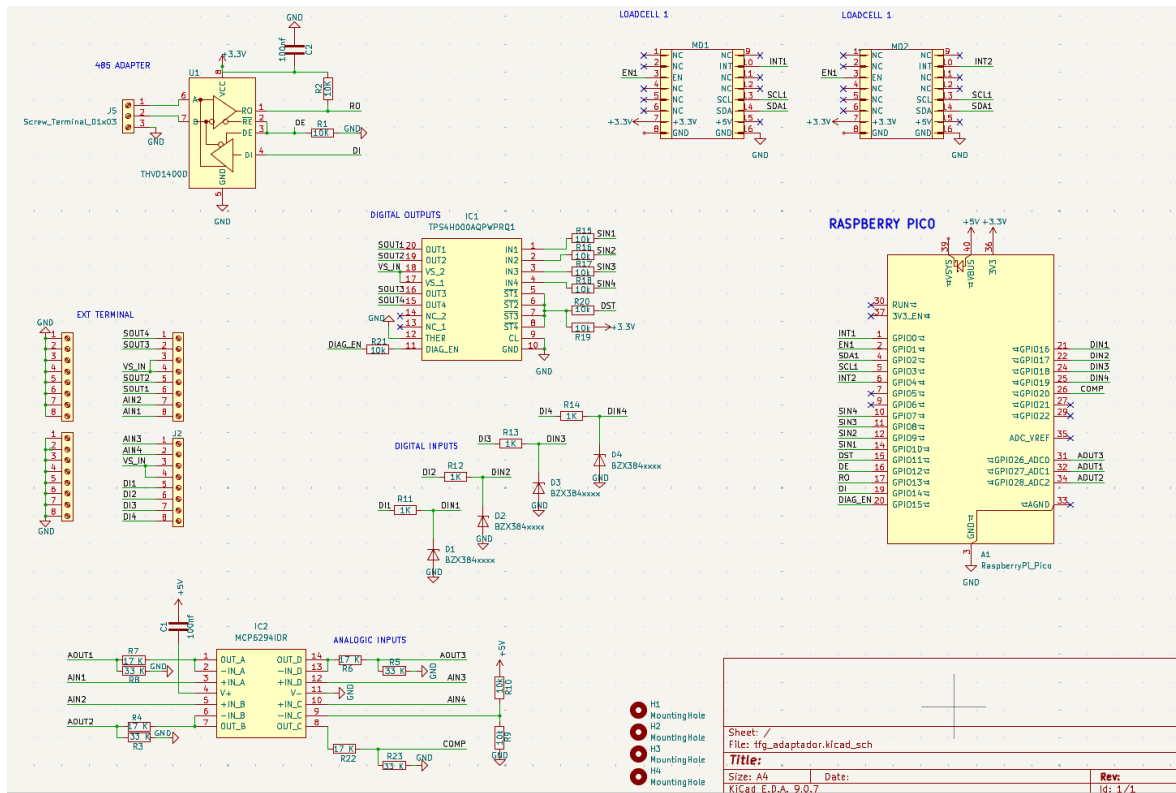


Figura 5.1: Esquema de la placa adaptadora, revisión B.

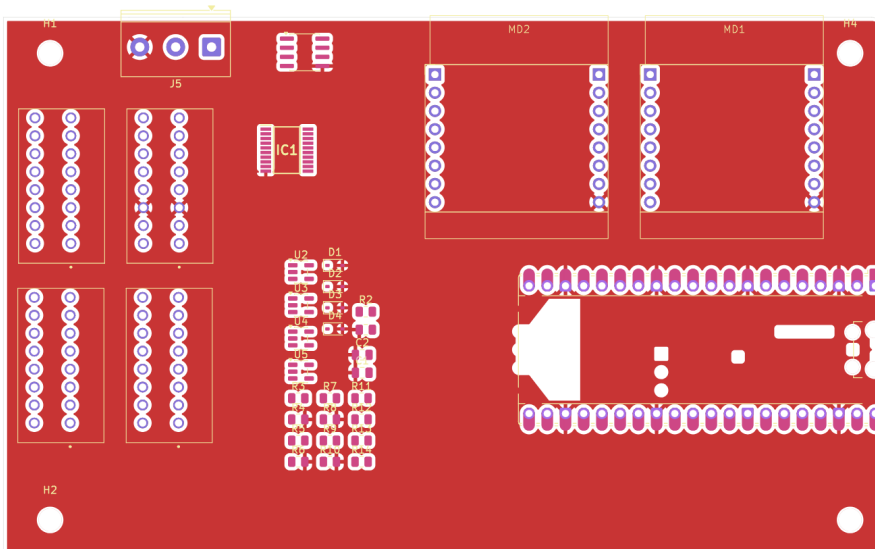


Figura 5.2: PCB de la revisión A: capas de cobre, máscara y serigrafía superiores.

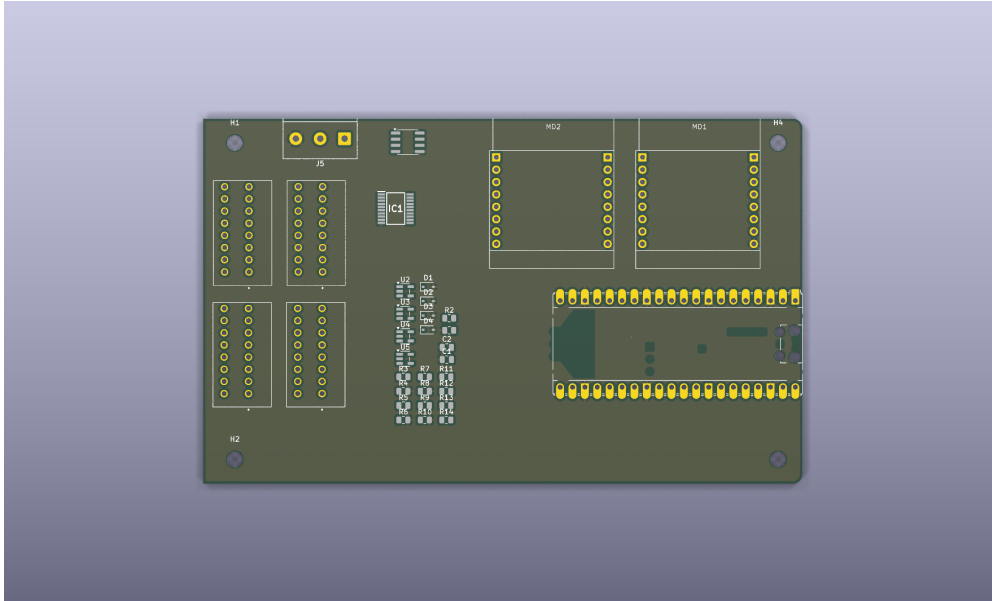


Figura 5.3: Vista 3D de la PCB de la revisión A generada con KiCad.

5.11. Validación de la placa

Cada bloque se validó con un programa de prueba independiente antes de la integración (tabla 5.2).

Tabla 5.2: Pruebas de la placa adaptadora.

Programa	Procedimiento y criterio	Resultado
pcbtest/test_analog	Llevar AIN a GND, 3,3 V y 5 V; la tensión reconstruida debe ser 0, 3,3 y 5,0 V.	[COMPLETAR: lecturas]
pcbtest/test_din	Cambiar el estado de DIN1–4 y comprobar que se detecta cada flanco.	[COMPLETAR:]
pcbtest/test_digital_outputs	Conmutar SIN1 cada segundo y medir la salida.	[COMPLETAR:]
test_limit_switches	Pulsar cada final de carrera: nivel bajo en reposo y alto activado.	Correcto (usado en todos los ensayos)
test_rs485_loopback	Puente GP14–GP13 sin transceptor: debe recibirse la trama enviada a 9600 y 115200 baudios.	Correcto
test_rs485_tx/rx	Transmisión y recepción a través del transceptor.	Correcto
test_modbus_scan	Lectura periódica del registro 0 del esclavo 1 a 115200 baudios.	Correcto (código 0x00)

Programa	Procedimiento y criterio	Resultado
test_loadcell	Detección del ZSC31014 en 0x28 y lectura del puente con estado.	Correcto

Capítulo 6

Firmware del microcontrolador

6.1. Entorno y organización

El firmware se ha desarrollado en C++ con el entorno Arduino y el núcleo Arduino-Pico [28], que da acceso a los periféricos del RP2350 y a su SDK. Usa las bibliotecas `Wire` (I²C), `SerialPIO` (UART implementada con PIO) y `ModbusMaster` [29]. El código final está en `firmware_posicion_v2/`: un fichero principal de unas 1500 líneas y la cabecera `i2c_bus.h` con el árbitro del bus I²C.

El diseño sigue un principio que se repite en todo el código: *cada bus tiene un único dueño y un único punto del lazo donde se usa*. Sólo dos funciones tocan el bus RS-485 (`servoReadState` y `servoAction`) y sólo dos tocan el bus I²C durante el funcionamiento (`i2cUpdate` y `lcRequestService`). El intérprete de órdenes (`processCommand`) nunca accede a un bus: se limita a modificar variables de estado, que el lazo lee en el punto que corresponde. Así, la llegada de una orden no puede interrumpir una transacción a medias.

El estado está en dos estructuras:

- **ControlState** `ctrl`: estado de la aplicación (modo, errores, parámetros, célula de carga, posición, *homing* y ciclo).
- **Servo** `sv`: contexto del enlace con el accionamiento (fase, dirección, conexión, contadores de fallos, consigna pendiente y últimas lecturas).

6.2. Lazo principal

El núcleo 0 ejecuta un lazo con periodo mínimo de 50 ms (`LOOP_MS`). Cada vuelta tiene cinco fases (listado 6.1).

```
1 void loop(){
2   unsigned long now = millis();  uint32_t t0us = micros();
3
4   // == LECTURA ==
5   servoReadState();              // RS-485: velocidad y par (2 transacciones)
6   lcRequestService();            // consulta/programación de la célula si se ha pedido
7   i2cUpdate();                  // I2C: una lectura de la célula
8   positionUpdate();             // integra rpm -> posición (sin buses)
9
10  // == LÓGICA == (sin acceso a buses)
11  checkCommands();               // órdenes USB -> variables de estado
12  if (ctrl.lcProgBusy)           { sv.speedCmd = 0; ctrl.cycPhase = CYC_IDLE; }
13  else if (homing en curso)      homingUpdate();
14  else                            { homingUpdate(); cycleUpdate(); }
```

```

15 // protecciones: límite de fuerza y finales de carrera anulan la consigna
16 // derivación del estado de la aplicación para la telemetría
17
18 // == ACTUACIÓN ==
19 servoAction();           // RS-485: máquina de estados del servo y escritura de consigna
20
21 // == REGISTRO ==
22 recordSample(now, micros() - t0us);
23 if (sampleCount >= MAX_SAMPLES || now - ctrl.lastTelemetryTime >= 200)
24     telemetryFlush();    // + tramas de homing y de ciclo si están activos
25
26 // == RITMO ==
27 unsigned long dt = millis() - now;
28 if (dt < LOOP_MS) delay(LOOP_MS - dt);
29 }

```

Listado 6.1: Estructura del lazo principal (simplificada).

Este orden tiene consecuencias deliberadas:

- Las protecciones se evalúan *después* de la lógica y *antes* de la escritura. Ninguna consigna generada por el *homing*, el ciclo o una orden manual puede llegar al accionamiento sin pasar por ellas.
- Una orden recibida se aplica en la misma vuelta, porque la escritura de la consigna es posterior a la lectura de órdenes.
- El tiempo de trabajo (*work_us*) se mide antes del retardo de ritmo y viaja con cada muestra. Es la base de la caracterización temporal de la sección 10.2.
- El retardo final hace que el periodo no dependa de la duración del trabajo mientras ésta no supere 50 ms.

El núcleo 1 no se usa (*loop1()* sólo espera). La evolución desde la versión de doble núcleo se explica en la sección 9.1.

6.3. Máquina de estados del servo

La figura 6.1 muestra la máquina de estados del enlace con el accionamiento, que ejecuta *servoAction*.

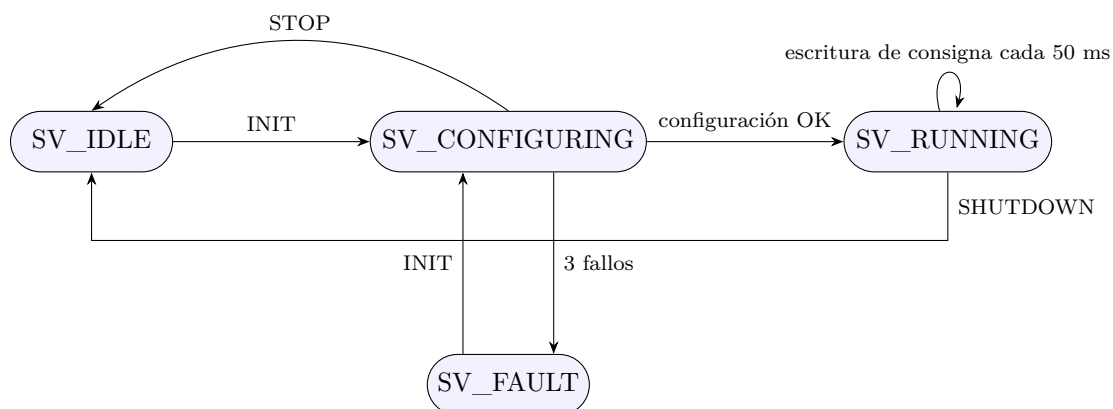


Figura 6.1: Máquina de estados del enlace con el accionamiento.

En `SV_CONFIGURING` se ejecuta la secuencia de configuración de la sección 7. En `SV_RUNNING` cada vuelta escribe la consigna: un fallo marca el servo como desconectado y pone a cero las lecturas, pero no provoca un nuevo escaneo del bus, que es bloqueante y lento (hasta un segundo por dirección vacía). El escaneo sólo se hace bajo demanda con la orden `SCAN`. El estado que se publica en la telemetría se deriva del de esta máquina y de la consigna (`STOPPED`, `MOVING_A` o `MOVING_B`). El *homing* y la programación de la célula tienen prioridad sobre él.

6.4. Lectura de la célula de carga

El ZSC31014 trabaja en modo de conversión continua: convierte a su propio ritmo, fijado por la palabra `ZMDI_Config1`, de 0,5 a 125 ms según el reloj y la tasa elegidos [21], y guarda el último resultado. Una lectura no dispara una conversión; devuelve el último dato con dos bits de estado que indican si es nuevo o si ya se había leído.

Por ello, `loadCellUpdate` hace *una sola lectura de dos bytes por vuelta*:

1. Pide el bus al árbitro. Si está ocupado, pierde el turno y lo intenta en la vuelta siguiente.
2. Lee 14 bits de dato y 2 de estado. Si el estado es “dato válido”, lo introduce en una media móvil de 8 posiciones y calcula la lectura tarada `baseRead = media - zeroOffset`.
3. Si pasan más de 150 ms sin un dato válido, marca la célula como no válida. El límite de fuerza deja entonces de actuar, en lugar de hacerlo con un dato viejo.

6.4.1. Árbitro del bus I²C

La cabecera `i2c_bus.h` implementa un árbitro mínimo: una variable con el dueño actual (`I2C_FREE`, `I2C_LC1`, `I2C_LC2` o `I2C_CFG`) y dos funciones, `i2cAcquire` e `i2cRelease`. La reserva se deniega si el bus tiene otro dueño o si se pide desde el núcleo 1, y cada denegación se cuenta para el diagnóstico. El árbitro nació al compartir el bus entre el ZSC31014 y el NAU7802 (sección 9.5), y se conserva para reservar el bus en exclusiva durante la programación de la EEPROM. Nadie espera con el bus ocupado, así que el árbitro no puede bloquear el lazo.

6.5. Posición integrada

La posición del eje del motor se obtiene integrando la velocidad que devuelve el accionamiento:

$$p_k = p_{k-1} + \frac{n_k \Delta t_k}{60000}, \quad [p] = \text{rev}, [n] = \text{rpm}, [\Delta t] = \text{ms}. \quad (6.1)$$

Es una integración rectangular con la última velocidad leída y el intervalo real medido con `millis()`. Hay que tener en cuenta tres aspectos:

- **Rebase tras un hueco.** Si el lazo se detiene, por ejemplo durante la programación de la célula, la siguiente vuelta vería un Δt de casi un segundo y lo multiplicaría por la última velocidad, inventando un desplazamiento que no ha ocurrido. Por eso se anula la base de tiempo (`lastPosTime = 0`) tras `INIT`, al iniciar el *homing* y tras cualquier operación bloqueante.

- **Resolución.** El registro de velocidad tiene resolución de 1 rpm. Si el accionamiento redondea, el error es de media nula; si trunca, a 30 rpm introduciría un sesgo de hasta el 3%. La dispersión observada en la posición del final B (13 mrev en 1,75 rev) indica que el sesgo real es menor.
- **Acotación.** La deriva no se acumula entre ensayos, porque el *homing* fija el origen en un final de carrera y cada ciclo termina con referencia a él.

6.6. Homing

La rutina de *homing* (figura 6.2) establece un origen repetible y mide el recorrido disponible:

1. **Buscar A:** avanza hacia A a la velocidad de *homing* hasta que se activa el final A, y fija ahí $p = 0$.
2. **Buscar B:** avanza hacia B hasta el final B y guarda el recorrido p_B .
3. **Ir al origen:** se mueve hasta el desplazamiento configurado, acotado a $[0, p_B]$, con tolerancia de $\pm 0,02$ rev. También da la llegada por buena si toca el final de carrera en el sentido de la marcha.
4. Fija ese punto como nuevo origen ($p = 0$) y sigue informando del resultado durante 1,5 s para que la interfaz lo muestre.

Cualquier tramo que dure más de 30 s, o la pérdida del servo, aborta con fallo. Durante el *homing* se ignoran las órdenes de movimiento manual.

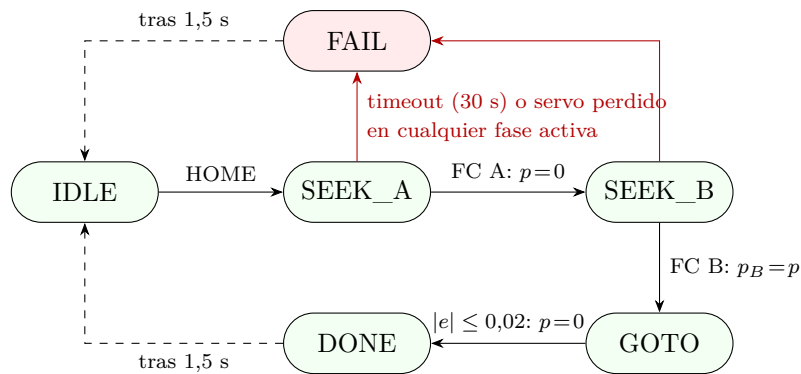


Figura 6.2: Máquina de estados del *homing*.

6.7. Ciclo automático A↔B

El ciclo lleva el eje del origen (A, $p = 0$) al punto B y vuelve, tantas veces como se pida (0 = sin fin):

- **Hacia B:** consigna $+v$ hasta que se activa el final B o, si B es una posición, hasta $p \geq p_B - 0,02$. El final B actúa siempre como tope de respaldo.
- **Hacia A:** consigna $-v$ hasta $p \leq 0,02$ o hasta que se activa el final A. Al llegar se incrementa el contador y se decide si se termina o se empieza otra repetición.

El ciclo no hace un control fino de posición: manda velocidad y observa la posición integrada, igual que el *homing*. Sólo se acepta con el servo habilitado, sin *homing* en curso y sin programación de la célula. Los temporizadores y la pérdida del servo lo abortan, y los finales de carrera y el límite de fuerza siguen actuando en el lazo. La prioridad es: programación de la célula > *homing* > ciclo > movimiento manual.

6.8. Configuración del acondicionador de la célula

La consulta y la programación de la EEPROM del ZSC31014 son las operaciones más delicadas del firmware. Por las restricciones que se analizan en la sección 9.2, obligan a reiniciar el chip y dejan la célula sin medida durante cientos de milisegundos. Se ejecutan así:

1. `processCommand` sólo *anota* la petición y un plazo de 1 s.
2. En su punto fijo del lazo, `lcRequestService` comprueba que el eje está en reposo: estado IDLE o STOPPED, consigna nula y sin *homing*. Si no lo está, responde “rechazado”.
3. Reserva el bus en exclusiva (I2C_CFG). Si no lo consigue antes del plazo, responde “bus ocupado”.
4. Pone la consigna a cero, pasa al estado PROGRAMMING y envía la telemetría pendiente para que la interfaz vea el silencio que viene.
5. Ejecuta la consulta o la programación (siguiente apartado), devuelve el bus a 400 kHz, libera el árbitro, reinicializa la célula, anula la base de la integración de posición y responde con el paquete 0x04.

6.8.1. Escritura verificada

`loadCellProgram` aplica dos reglas de la hoja de datos [21]: la palabra ZMDI_Config1 sólo se carga tras un ciclo de alimentación, y la firma MISR de la EEPROM (palabra 0x12) se regenera al salir del modo comando con Start_NOM si hubo escrituras. La función hace dos pasadas:

1. **Primera pasada:** entra en modo comando, lee B_Config (0x0F) y ZMDI_Config1 (0x01) y calcula las palabras deseadas. Si coinciden con las actuales, termina sin escribir, porque la EEPROM admite 100.000 ciclos de escritura. Si no, escribe las palabras que cambian, espera 20 ms por escritura (típico 15 ms, peor caso 17,3 ms), envía Start_NOM y espera 150 ms antes de cortar la alimentación (típico 15 ms).
2. **Segunda pasada:** desde un reinicio limpio, vuelve a leer ambas palabras. Si coinciden con lo pedido, el resultado es “grabado y verificado”; si no, “fallo”.

Las palabras deseadas se construyen a partir de las actuales, protegiendo los bits que la interfaz no debe tocar:

- B_Config: se conservan los bits reservados [15:13]; se aplican anulación, multiplexor, B-sink, integración larga, polaridad, ganancia [6:4] y cero del A/D [3:0].
- ZMDI_Config1: se conservan [15:9] (signos de las correcciones de segundo orden y de temperatura, calibrados en fábrica); se aplican tasa [7:6], modo de bajo consumo [5] y reloj [3]; se fuerzan el interfaz I²C (bit 4 = 0) y el campo reservado [2:0] = 001.

6.8.2. Mantenimiento del reinicio para medida

La orden `LC_HOLD` deja la célula sin alimentación durante 8 s, con las líneas del bus en el estado elegido, para medir con un polímetro la tensión de la placa Click. A diferencia de la programación, esta espera *no* bloquea: el lazo sigue girando, leyendo el servo, vigilando los finales de carrera y enviando telemetría, y en cada vuelta sólo comprueba si ha vencido el plazo. Mientras dura, se fuerza la consigna a cero en cada vuelta: alguien tiene las manos dentro de la máquina, y una orden de movimiento que llegue por USB no debe poder arrancar el eje.

6.9. Evolución del firmware

Tabla 6.1: Versiones del firmware conservadas en el repositorio.

Versión	Placa	Características
<code>old/speedtorque_mode</code>	XIAO	Escaneo Modbus y selección de modo par o velocidad.
<code>firmware</code>	ESP32-S3	
	XIAO	Protocolo binario v2, parámetros del accionamiento (rigidez, límites de par, resistencia de frenado) y regulador de fuerza en modo par con zona muerta, umbral y ganancia sobre la célula filtrada (media móvil de 12).
<code>firmwarerasp</code>	Pico 2	Migración con SerialPIO. Doble núcleo: el núcleo 1 escribe la consigna y lee la supervisión Modbus en bucle, comunicado con el núcleo 0 por variables <code>volatile</code> y un protocolo de relevo (<code>active/busy</code>).
<code>firmware_posicion</code>	Pico 2	Modo velocidad A→B sin lazo de fuerza, integración de posición, telemetría por lotes y diseño de dueño único en el núcleo 0.
<code>firmware_posicion_v2</code>	Pico 2	Versión final: <i>homing</i> , ciclos, finales de carrera, programación verificada de la EEPROM de la célula, árbitro I ² C, tiempo de trabajo en la telemetría y respuestas de diagnóstico.

Capítulo 7

Comunicaciones

El sistema usa cuatro enlaces (tabla 7.1). Este capítulo describe el protocolo binario propio entre el microcontrolador y el ordenador, el uso de Modbus RTU con el accionamiento y la interfaz entre el servidor y el navegador. La especificación byte a byte de todas las tramas está en el anexo B.

Tabla 7.1: Enlaces de comunicación del sistema.

Enlace	Extremos	Capa física	Protocolo
USB	PC ↔ Pico	USB CDC (puerto serie virtual)	Binario propio + CRC-8
RS-485	Pico ↔ accionamiento	UART PIO, 115200 8N1	Modbus RTU
I ² C1	Pico ↔ ZSC31014	400 kHz (100 kHz en modo comando)	Propio del chip
Red local	Navegador ↔ servidor	TCP, puerto 8080	HTTP + WebSocket (JSON)

7.1. Protocolo binario entre el microcontrolador y el ordenador

7.1.1. Formato de trama

Todas las tramas, en ambos sentidos, tienen el mismo formato (figura 7.1): dos bytes de sincronía 0xAA 0x55, un byte de longitud, la carga útil y un CRC-8 calculado sobre la carga útil. El primer byte de la carga útil identifica el tipo de paquete o la orden. Los enteros multibyte van en orden *little-endian*.

0xAA	0x55	LEN	ID	datos (LEN-1)	CRC-8
------	------	-----	----	---------------	-------

Figura 7.1: Formato común de trama. El CRC-8 cubre los LEN bytes de la carga útil (ID + datos).

El CRC usa el polinomio $x^8 + x^2 + x + 1$ (0x07), valor inicial 0x00, sin reflexión ni XOR final (la variante conocida como CRC-8/SMBUS). Se implementa bit a bit en el firmware y en Python con el mismo algoritmo. Para tramas cortas como éstas detecta todos los errores de uno y dos bits y todas las ráfagas de hasta 8 bits.

7.1.2. Recepción y resincronización

El firmware decodifica las órdenes con una máquina de cuatro estados (RX_SYNC0, RX_SYNC1, RX_LEN, RX_PAYLOAD). Una longitud nula o mayor de 60 devuelve el decodificador a la búsqueda de sincronía, y una trama con CRC incorrecto se descarta sin ejecutarse. Si se pierden bytes, el receptor vuelve a encontrar el patrón AA 55 en la siguiente trama. El servidor hace lo mismo con la telemetría, con un tiempo de espera de 100 ms por lectura.

Para depurar desde un terminal, el firmware acepta también órdenes de una letra (i, a, b, x, h, q), pero *sólo* cuando el decodificador está esperando el primer byte de sincronía. Antes se comprobaban en cualquier estado, y eso provocaba un error muy difícil de ver: la orden de programar la célula con ganancia 7 y cero 8 genera la trama AA 55 03 09 07 08 69, cuyo CRC 0x69 coincide con el carácter i. El atajo se comía el último byte, la orden se perdía y además se reiniciaba el servo. Con ganancia 6 el CRC vale 0x7C y la misma orden funcionaba. Ambos valores se han comprobado con la implementación del CRC.

7.1.3. Paquetes del microcontrolador al ordenador

Tabla 7.2: Paquetes de telemetría y estado.

ID	Nombre	Tamaño de carga útil	Cuándo se envía
0x02	Telemetría por lotes	14 + 14n B, $n \leq 8$	cada 200 ms o con 8 muestras
0x03	Estado de <i>homing</i>	11 B	junto al lote, mientras hay <i>homing</i>
0x04	Resultado de la célula	13 B	una vez por consulta o programación
0x05	Estado del ciclo	14 B	junto al lote, mientras hay ciclo

Telemetría por lotes. Cada vuelta del lazo añade al búfer una muestra de 14 bytes: desfase en milisegundos respecto al instante base del lote, velocidad, par, consigna, fuerza tarada, estado de los finales de carrera, estado de la máquina y tiempo de trabajo de la vuelta en microsegundos. Los datos comunes (instante base de 32 bits, lectura bruta de la célula y su estado, indicadores, código de error, dirección del servo y códigos de resultado de las dos lecturas Modbus) viajan una sola vez en una cabecera de 14 bytes.

Agrupar las muestras reduce las escrituras por USB a cinco por segundo sin perder resolución temporal, porque cada muestra conserva su instante exacto. Con el lazo de 50 ms cada lote lleva típicamente cuatro muestras: 70 bytes de carga útil, 74 de trama y unos 370 B/s en total, muy por debajo de la capacidad del enlace. El precio es un retardo de visualización de hasta 200 ms, irrelevante para la supervisión, y que la lectura bruta de la célula sólo se refresque una vez por lote (sección 10.9).

Modelo pregunta-respuesta de la célula. Cada orden de consulta, programación o mantenimiento de la célula produce exactamente un paquete 0x04, también cuando se rechaza. El paquete incluye el código de la orden a la que responde, para que la interfaz nunca se quede esperando. Además de la ganancia, el cero y las palabras de configuración antes y después, lleva información de diagnóstico: el primer byte que devolvió el chip, si el pin de reinicio corta de verdad la alimentación y en qué intento del barrido de retardos se consiguió entrar en modo comando (sección 9.2).

7.1.4. Órdenes del ordenador al microcontrolador

Tabla 7.3: Órdenes aceptadas por el firmware.

Código	Nombre	Datos	Efecto
0x01	INIT	[dirección Modbus] opcional	Reinicia la posición y configura y habilita el servo.
0x02	STOP	–	Consigna a cero; aborta <i>homing</i> y ciclo.
0x03	SET_PARAM	id (1 B), valor int16	Modifica un parámetro (tabla 7.4).
0x04	MOVE_A	–	Movimiento manual hacia A.
0x05	MOVE_B	–	Movimiento manual hacia B.
0x06	SHUTDOWN	–	Deshabilita el accionamiento.
0x07	SCAN	–	Búsqueda bloqueante del esclavo (1–247).
0x08	HOME	–	Inicia el <i>homing</i> .
0x09	LC_PROGRAM	B_Config, ZMDI_Config1 (uint16)	Graba la configuración de la célula (sólo parado).
0x0A	LC_QUERY	–	Lee la configuración de la célula (sólo parado).
0x0B	LC_HOLD	modo de aparcado (1 B)	Mantiene la célula sin tensión 8 s para medirla.
0x0C	CYCLE_START	modo, objetivo (centi-rev), velocidad, repeticiones	Inicia el ciclo A↔B.
0x0D	CYCLE_STOP	–	Detiene el ciclo donde esté.

Tabla 7.4: Parámetros configurables con SET_PARAM.

Id	Parámetro	Unidad	Defecto
1	Velocidad de movimiento manual	rpm	10
2	Límite de fuerza (0 = desactivado)	cuentas	0
3	Tara de la célula	cuentas	8210
4	Aceleración (C03.22)	u. del accionamiento	5
5	Deceleración (C03.24)	u. del accionamiento	5
6	Rigidez (C00.05)	nivel	10
7	Calibración de fuerza (sólo informativa)	cuentas/N	0
8	Desplazamiento del origen tras el <i>homing</i>	centi-rev	0
9	Velocidad de <i>homing</i>	rpm	30

7.2. Modbus RTU con el accionamiento

7.2.1. Registros utilizados

El firmware sólo usa las funciones 0x03 y 0x06 sobre un conjunto reducido de registros (anexo C). Los parámetros del accionamiento se nombran `Cgg.ii`, y todas las direcciones del firmware cumplen la regla

$$\text{dirección} = 256 \cdot gg + 0xii, \quad (7.1)$$

es decir, el índice se interpreta en hexadecimal. Por ejemplo, C04.11 (habilitación) es $1024 + 17 = 1041$ y C03.22 (aceleración) es $768 + 34 = 802$. Las magnitudes de supervisión están en el grupo 0x40: velocidad en 16385 (0x4001) y par en 16387 (0x4003). Tener esta regla evita errores al añadir registros a partir del manual.

7.2.2. Secuencia de configuración

Al recibir INIT, el firmware configura el accionamiento con esta secuencia: deshabilitar (C04.11 = 0), seleccionar el modo velocidad (C00.00 = 1), poner la consigna a cero (C03.21 = 0), escribir la rigidez y las rampas, configurar la resistencia de frenado externa (C00.10 a C00.13) y habilitar (C04.11 = 1). Si una escritura esencial falla se reintenta la secuencia completa, y tras tres fallos el sistema pasa a estado de error. Entre escrituras se deja un silencio de 700 µs.

7.2.3. Transacciones por periodo

En régimen normal, cada vuelta del lazo realiza tres transacciones: lectura de la velocidad, lectura del par y escritura de la consigna. Escribir la consigna en cada vuelta, aunque no cambie, sirve a la vez de supervisión del enlace: si la escritura falla, se marca el servo como desconectado y se cuentan los fallos consecutivos.

A 115200 baudios con formato 8N1, cada byte ocupa 86,8 µs. Las tres transacciones suman 46 bytes (8 + 7 por cada lectura y 8 + 8 por la escritura), unos 4,0 ms de línea. A eso se añaden 0,6 ms de retardos de conmutación del transceptor y 0,7 ms de silencio entre las dos lecturas. El tiempo de trabajo medido del lazo es 10,97 ms (sección 10.2). La diferencia, unos 5,7 ms, corresponde principalmente al tiempo de respuesta del accionamiento, aproximadamente 1,9 ms por transacción según esta estimación.

La guía de implementación de Modbus serie recomienda, por encima de 19200 baudios, silencios fijos de 1,75 ms entre tramas [11]. El firmware usa 700 µs y el accionamiento lo acepta sin errores, pero con otros esclavos convendría respetar el valor recomendado.

7.3. Interfaz entre el servidor y el navegador

7.3.1. WebSocket

El servidor mantiene una conexión WebSocket en `/ws` con cada navegador. En sentido servidor → navegador envía objetos JSON de cinco tipos:

- **muestra de telemetría** (sin campo `type`): una por muestra del lote, con los valores brutos, los nombres de estado y error ya traducidos, la posición integrada, los finales

de carrera, la frecuencia de muestreo medida y el estado del *homing*;

- **home**: fase del *homing*, posición y recorrido medido;
- **cycle**: fase, repeticiones hechas y pedidas y fichero en grabación;
- **lc_config**: resultado de la consulta o programación de la célula;
- **cmd_ack**: confirmación de *toda* orden recibida, con indicación de éxito y un texto explicativo. Sin ella, una orden que no llegaba al puerto era indistinguible de una que sí se había ejecutado.

En sentido navegador → servidor, las órdenes son objetos `{"cmd": nombre, ...parámetros}`. El servidor las valida, limita los parámetros a rangos seguros (por ejemplo, velocidad del ciclo entre 0 y 300 rpm), construye la trama binaria y la escribe en el puerto.

7.3.2. Servicios HTTP

Tabla 7.5: Servicios HTTP del servidor.

Método	Ruta	Función
GET	/	Página de supervisión en tiempo real
GET	/ensayos	Visor de ensayos grabados
GET	/cycles	Lista de los 50 ensayos más recientes (JSON)
GET	/cycles/{nombre}	Descarga de un ensayo (CSV)
DELETE	/cycles/{nombre}	Borrado de un ensayo (rechazado si se está grabando)

Las rutas con nombre de fichero reducen el parámetro a su último componente antes de usarlo, de modo que una petición con `../` no puede leer ni borrar ficheros fuera de la carpeta de ensayos.

Capítulo 8

Software de supervisión y análisis

8.1. Visión general

El software del ordenador consta de un servidor en Python (`web_monitor.py`), dos páginas web (`static/index.html` para la operación en vivo y `static/ensayos.html` para el análisis) y varias herramientas auxiliares. El servidor se ejecuta con

```
python web_monitor.py [puerto_serie]      # http://localhost:8080
```

y detecta solo el puerto del microcontrolador si no se indica.

8.2. Servidor

8.2.1. Estructura concurrente

El servidor combina un hilo del sistema operativo para el puerto serie con el bucle asíncrono de FastAPI (figura 8.1).

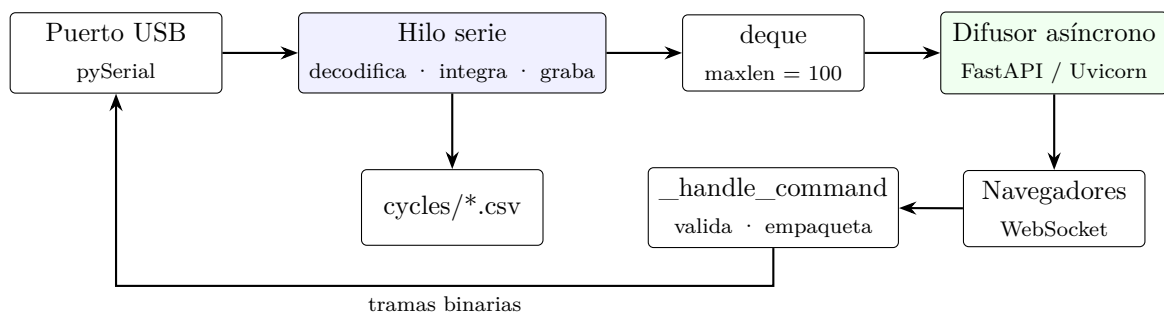


Figura 8.1: Estructura del servidor de supervisión.

Hilo serie. Abre el puerto (probando `/dev/ttyACM*`, `/dev/ttyUSB*`, `/dev/cu.usbmodem*` y `COM*`), vacía el búfer y entra en un bucle que lee tramas, comprueba el CRC y las decodifica con `struct`. Cualquier excepción cierra el puerto y provoca un reintento al cabo de un segundo, lo que cubre la desconexión física y el reinicio del microcontrolador al reprogramarlo (RNF4). Por cada muestra de telemetría:

- mide la frecuencia de muestreo contando las muestras del último segundo *con el reloj del dispositivo*, no con el del ordenador;
- detecta que el reloj del microcontrolador ha vuelto a cero (reinicio) y en ese caso reinicia la ventana de medida y la integración. Sin esta comprobación, la ventana no se podaba

nunca y la interfaz llegó a mostrar “3813 Hz”, que en realidad era el número de muestras acumuladas;

- integra la posición a partir de la velocidad, descartando saltos de tiempo anómalos;
- si hay un ensayo en grabación, escribe la fila correspondiente;
- deposita un mensaje JSON en la cola.

Cola y difusor. La comunicación entre el hilo y el bucle asíncrono usa una **deque** de la biblioteca estándar con un único productor y un único consumidor, segura sin cerrojos en CPython para **append** y **popleft**. La longitud máxima de 100 mensajes acota la memoria si no hay ningún navegador conectado: los mensajes más antiguos se descartan. El difusor envía cada mensaje a todos los clientes, elimina los que han cerrado la conexión y, cuando la cola está vacía, duerme 20 ms.

Estado compartido. Los valores que necesitan varios contextos (posición, calibración, estado del *homing*) se guardan en un diccionario protegido por un **threading.Lock**.

Registro. Todas las tramas, órdenes y confirmaciones se registran con marca de tiempo en consola y en **web_monitor.log**. El registro ha sido la principal herramienta de diagnóstico de los problemas de comunicación.

8.2.2. Grabación de ensayos

La grabación es automática y la dirigen los paquetes de ciclo: cuando la fase pasa a “hacia B” o “volviendo a A” y no hay fichero abierto, se crea **cycles/AAAAMDD-HHMMSS_ciclo.csv**; cuando la fase pasa a completado, abortado o reposo, se cierra. Cada fichero tiene una cabecera con las constantes del ensayo (calibración de la célula, fuerza en el punto de tara, repeticiones y punto B) y una fila por muestra (formato completo en el anexo D).

Se graban deliberadamente las *cuentas brutas* de la célula y no newtons. La calibración puede cambiar después del ensayo (de hecho, la de este trabajo es provisional), y con los datos brutos basta con aplicar la nueva constante (RNF5).

8.3. Página de supervisión

La página principal (**index.html**) se organiza en tres zonas:

1. **Estado:** indicador de conexión, estado de la máquina y código de error, modo, estado del servo y de la célula, tiempo del dispositivo y frecuencia de muestreo, finales de carrera con fase del *homing* y recorrido medido, y una tarjeta destacada con la fuerza sobre la carga.
2. **Controles en pestañas:**
 - *Servo:* dirección Modbus, inicialización, movimiento manual, *homing*, parada, puesta a cero de la posición y apagado; velocidad, desplazamiento del origen y velocidad de *homing*.

- *Célula 1*: calibración (cuentas/N), fuerza en el punto de tara y tara. En la configuración avanzada se accede a la palabra B_Config (ganancia de $\times 1,5$ a $\times 192$, cero del A/D, polaridad, puente completo o medio, integración larga, alimentación del puente y anulación de desequilibrio) y a ZMDI_Config1 (reloj de 4 o 1 MHz, tasa de conversión y modo continuo o a demanda), con el botón de programar. La configuración grabada se consulta automáticamente la primera vez que se abre la pestaña con el eje parado.
 - *Ciclo A $\leftrightarrow$ B*: punto B (final de carrera o posición), velocidad y repeticiones; arranque y parada; lista de ensayos y enlace al visor.
 - *Cálculo teórico*: esquema del mecanismo y curvas de tensión, par y ángulo para un tambor de radio constante, con comparación con el par medido.
3. **Gráficas**: velocidad consigna y real con los finales de carrera y marcas verticales en los instantes en que se enviaron órdenes, y fuerza frente al tiempo.

8.4. Visor de ensayos

El visor (`ensayos.html`) lista los ensayos grabados y, al abrir uno, muestra:

- un **resumen** con número de muestras, duración, cadencia, repeticiones y rangos de posición, par y fuerza, y una tabla por repetición;
- **fuerza frente a posición**, con una traza por repetición, donde se ven directamente la ida, la vuelta y la histéresis;
- **fuerza y posición frente al tiempo**, con la posibilidad de superponer la curva teórica del capítulo 3 para un tambor cilíndrico o una caracola lineal. Se introducen las cotas, la masa, el ángulo de la barra en el origen, la reducción y el sentido de giro;
- **par frente a posición y frente al tiempo**, con el par teórico $\tau = Tr(\varphi)$;
- **velocidad consigna y real** con los finales de carrera.

Al superponer la curva teórica, el visor comprueba dos consistencias que han resultado muy útiles para detectar datos de entrada erróneos:

1. *Escala*: compara el cable que paga el tambor durante el ensayo con el que puede absorber el mecanismo desde el ángulo inicial hasta la vertical en ese sentido. Si difieren más de un 15 %, avisa y sugiere la reducción o los radios que lo harían cuadrar.
2. *Ángulo*: con la ecuación (3.2) calcula el ángulo de la barra en el origen, en el extremo y al final del ensayo, y avisa si el modelo exigiría superar la vertical o si el ángulo inicial introducido no es alcanzable.

Por último, ofrece un **ajuste por mínimos cuadrados** de la recta que convierte las cuentas de la célula en la fuerza teórica, $F = a \cdot \text{cuentas} + b$, con

$$a = \frac{n \sum x_i y_i - \sum x_i \sum y_i}{n \sum x_i^2 - (\sum x_i)^2}, \quad b = \frac{\sum y_i - a \sum x_i}{n}, \quad (8.1)$$

e informa del coeficiente de determinación R^2 y de la *cobertura*, el porcentaje de muestras

donde la geometría tiene solución. Un R^2 alto con poca cobertura no garantiza un ajuste válido, y por eso la interfaz lo señala.

8.5. Herramientas auxiliares

- `leer_ciclo.py`: resumen de un ensayo en consola, usando sólo la biblioteca estándar, y exportación sin cabecera para hojas de cálculo.
- `monitor.py` y `monitor_posicion.py`: monitores de terminal de las primeras versiones del protocolo, con control por teclado.
- `test_servo_modbus.py`: barrido de velocidades y direcciones Modbus con un adaptador USB-RS485 y un CRC-16 propio, útil para identificar la configuración del accionamiento sin el microcontrolador.
- `memoria/figuras/generar_figuras.py`: genera todas las figuras de resultados y las cifras de esta memoria a partir de los ficheros de ensayo y del modelo.

8.6. Observaciones de calidad

Durante la redacción de esta memoria se revisó el software y se identificaron los siguientes puntos de mejora, que no afectan a los resultados:

- `requirements.txt` sólo declara `pyserial`; faltan `fastapi` y `uvicorn[standard]`, que se instalan a mano.
- La cabecera del CSV decide si B es “posición” o “final de carrera” según el objetivo en revoluciones sea mayor que cero, no según el modo realmente pedido. Como la interfaz envía siempre el objetivo (1 rev por defecto), todos los ensayos de este trabajo aparecen como “posición” aunque se hicieron hasta el final de carrera. Los datos no se ven afectados; la corrección es incluir el modo en el paquete de ciclo.
- El servidor escucha en todas las interfaces (0.0.0.0) sin autenticación, así que cualquier equipo de la red puede mover la máquina. En un entorno compartido debería escuchar sólo en 127.0.0.1 o exigir credenciales.
- La posición se integra dos veces, en el firmware y en el servidor, con el mismo método. El CSV guarda la del servidor. Sería más limpio transmitir la del firmware en cada muestra.
- Algunos comentarios han quedado desfasados respecto al código: la cabecera del firmware todavía describe la versión de doble núcleo y el tamaño de muestra de 12 bytes.

Capítulo 9

Problemas técnicos y soluciones

Este capítulo reúne los problemas más costosos del desarrollo. Para cada uno se describen el síntoma, la causa y la solución, tal como quedaron documentados en el código y en el repositorio. Muchos de ellos no se deben a un error de programación, sino a detalles de los componentes que no son evidentes en una primera lectura de su documentación.

9.1. UART sobre PIO y latencia del enlace Modbus

Síntoma. Tras migrar el firmware del ESP32-S3 a la Pico 2, las órdenes tardaban entre 3 y 4 s en tener efecto.

Causa. En el ESP32-S3 la UART es un periférico hardware: los bytes de la respuesta del esclavo llegan a una FIFO por interrupción, y cuando ModbusMaster los pide ya están disponibles, así que cada transacción dura 1 a 3 ms. En la Pico, el trazado de la placa lleva la transmisión a GP14 y la recepción a GP13, y se usó `SerialPIO`, una UART implementada con una máquina de estado PIO que funciona en cualquier pin [\[VERIFICAR: confirmar si el RP2350 permite la función UART en GP14 mediante su función alternativa\]](#). En la práctica, con `SerialPIO` el bucle de espera de la respuesta de ModbusMaster agotaba su tiempo límite, de unos 20 ms, en muchas transacciones. Diez escrituras consecutivas bloqueaban así el núcleo 200 ms, y al acumular las lecturas de supervisión se llegaba a varios segundos.

Un segundo detalle afectaba al cambio de sentido del transceptor. Llamar a `flush()` antes de desactivar el emisor, como se haría con una UART hardware, rompía la recepción inmediata de la respuesta con `SerialPIO`. La solución fue esperar un tiempo fijo de 200 µs tras el último byte y desactivar entonces `DE/RE`.

Primera solución: doble núcleo. La versión `firmwarerasp` dedicó el núcleo 1 al bus Modbus. El núcleo 0 publicaba la consigna en variables `volatile` y el núcleo 1 la escribía en bucle, leyendo velocidad y par cada diez escrituras. Para tomar el bus desde el núcleo 0 (por ejemplo, para configurar el servo), había que desactivar el trabajador y esperar a que su indicador `busy` bajara. El núcleo 0 quedaba libre, pero el diseño tenía inconvenientes: variables compartidas sin atomicidad, un protocolo de relevo con esperas, y lecturas del servo cuyo instante no estaba ligado al de la muestra de telemetría.

Solución final: dueño único con lazo de periodo fijo. Una vez ajustados los tiempos de espera y el cambio de sentido, cada transacción volvió a durar unos pocos milisegundos. La versión final prescinde del segundo núcleo: un único lazo de 50 ms hace exactamente tres

transacciones por vuelta, en posiciones fijas. El resultado es un tiempo de trabajo de 11,0 ms con sólo 0,2 ms de dispersión (sección 10.2), sin condiciones de carrera, y con lecturas, lógica y escritura en la misma vuelta y con la misma marca de tiempo.

9.2. Configuración del ZSC31014: la ventana del modo comando

Contexto. Para cambiar la ganancia o el cero del acondicionador hay que escribir en su EEPROM, y para ello hay que entrar en modo comando. Según la hoja de datos [21], tras un reinicio por encendido (POR) el chip espera la orden `Start_CM` durante una ventana de 6 ms si la EEPROM no está bloqueada, o de 1,5 ms si lo está. Pasada la ventana, entra en funcionamiento normal e ignora `Start_CM` sin dar ninguna señal de error.

9.2.1. Primer obstáculo: provocar el reinicio

Síntoma. Tras reprogramar la Pico o reiniciarla por USB, la configuración nunca se aplicaba.

Causa. La placa Click seguía alimentada: sin un POR no se abre la ventana.

Solución. Controlar la alimentación de la Click con su pin EN (GPIO1) y hacer un ciclo completo de apagado y encendido antes de cada intento.

9.2.2. Segundo obstáculo: alimentación parásita por el bus

Síntoma. Con EN a nivel bajo, el chip seguía respondiendo por I²C. La tensión de la Click, medida con un polímetro, se quedaba en unos 2,5 a 2,7 V en lugar de caer a cero.

Causa. Las resistencias de *pull-up* de SDA, SCL e INT cuelgan del 3,3 V permanente del zócalo mikroBUS, no del carril gobernado por EN. Con el chip sin alimentar, la corriente entraba por esos pines, atravesaba los diodos de protección internos y alimentaba el carril del chip. La hoja de datos sitúa el umbral de POR entre 1,8 y 2,5 V: con 2,5–2,7 V de alimentación parásita, el chip nunca veía un reinicio.

Diagnóstico. Para confirmarlo se añadió la orden `LC_HOLD`, que mantiene la célula sin alimentación 8 s sin bloquear el lazo, con dos modos para las líneas del bus: en alta impedancia o forzadas a nivel bajo. Las medidas mostraron que sólo forzándolas a nivel bajo la tensión caía a cero. Hubo además un error en el propio diagnóstico: la primera versión de la prueba del pin de reinicio dejaba las líneas en alto mientras medía, así que siempre “detectaba” realimentación y acusaba al pin EN sin motivo. Se corrigió para que midiera en exactamente las mismas condiciones que el ciclo real.

Solución. Durante el apagado se libera el periférico I²C y SDA, SCL e INT se configuran como salidas a nivel bajo (`LC_PARK_DEFAULT = 1`). El bus se restaura *antes* de volver a dar tensión, porque `Wire.begin()` tarda lo bastante como para comerse la ventana.

9.2.3. Tercer obstáculo: acertar la ventana

Síntoma. Incluso con un POR limpio, Start_CM no siempre entraba.

Causa. La orden tiene que llegar *después* de que el chip termine de arrancar y *antes* de que se cierre la ventana. El tiempo de arranque depende de los condensadores y del conmutador de alimentación de la placa Click, de modo que los retardos copiados de otros controladores (por ejemplo, los 500 µs de un controlador de mbed) no servían. Además, cualquier operación entre el encendido y Start_CM, aunque sea una impresión por el puerto serie, desplaza el instante de envío.

Solución. En lugar de adivinar el retardo, se mide:

1. tras dar tensión, se sondea la dirección del chip hasta que responde (unos 90 µs por sondeo), lo que marca el fin del arranque;
2. se espera un retardo que cambia en cada intento, de 0 a 1050 µs en pasos de 150 µs, a lo largo de hasta ocho ciclos de alimentación;
3. se envía Start_CM y se confirma la entrada leyendo B_Config, cuya respuesta debe empezar por 0x5A;
4. el intento que funciona se devuelve a la interfaz (paquete 0x04), para poder fijar el retardo más adelante.

Si ningún intento funciona, se clasifica la causa: “no contesta” si nunca hubo ACK, “el reinicio no corta” si sigue respondiendo con EN a nivel bajo, o “no entra en la ventana”.

9.2.4. Cuarto obstáculo: integridad de la EEPROM

La firma MISR que protege la EEPROM se recalcula al salir del modo comando con Start_NOM, y se comprueba en el siguiente POR. Si se corta la alimentación antes de que la firma quede grabada, el chip arranca desde entonces en estado de diagnóstico y la placa queda inservible. Por eso, tras cualquier escritura se envía Start_NOM y se espera diez veces el tiempo típico antes de cortar la tensión. Como ZMDI_Config1 sólo se carga tras un POR, la verificación se hace en una segunda pasada, desde un reinicio limpio.

El firmware fuerza siempre el interfaz I²C en ZMDI_Config1. Si se grabara SPI, el chip dejaría de responder por I²C en funcionamiento normal. La hoja de datos indica que tras el POR el chip atiende siempre por I²C durante la ventana del modo comando, de modo que sería recuperable, pero es mejor no correr el riesgo.

9.3. Lecturas de la célula siempre “antiguas”

Síntoma. En una versión del firmware no se registraba ni una sola muestra válida de la célula.

Causa. La lectura se hacía en dos fases, separadas 2 ms, quedándose con la segunda. En modo de conversión continua leer no dispara una conversión, y el chip no había producido un dato nuevo en 2 ms, así que la segunda lectura venía siempre marcada como repetida. Una

versión anterior funcionaba por casualidad: sus dos lecturas caían en vueltas distintas del lazo, separadas 50 ms.

Solución. Una única lectura por vuelta, aceptada sólo si el dato es nuevo. El ritmo efectivo es el menor entre el del lazo y el de conversión del chip.

9.4. Discrepancias con la hoja de datos detectadas durante la redacción

Al contrastar el firmware y la interfaz con la hoja de datos del ZSC31014 para esta memoria, se encontraron dos discrepancias que no afectan a los ensayos presentados, pero que conviene corregir:

1. **Bits de estado.** Según la hoja de datos, 00 = dato válido, **01 = modo comando**, **10 = dato repetido** y 11 = diagnóstico. El firmware define `LC_STATUS_STALE = 1` y `LC_STATUS_COMMAND = 2`, es decir, intercambiados. En la lectura normal sólo se acepta 00, así que no hay efecto. En la confirmación del modo comando, `zscResp` acepta un primer byte con estado 10 o igual a `0x5A`. La respuesta real es `0x5A`, cuyos bits altos son 01, y el firmware la acepta por la segunda condición; la primera condición es errónea pero inocua.
2. **Codificación de la ganancia.** Según la tabla de `B_Config`, el campo `PreAmp_Gain [6:4]` no es lineal: 000 = $\times 1,5$, 100 = $\times 3$, 001 = $\times 6$, 101 = $\times 12$, 010 = $\times 24$, 110 = $\times 48$, 011 = $\times 96$ y 111 = $\times 192$. El índice de ganancia corresponde a los tres bits en orden inverso. El selector de la interfaz supone un código lineal (1 = $\times 3$, 2 = $\times 6$...), así que para los códigos 1 a 6 la ganancia grabada no es la mostrada: elegir “ $\times 3$ ” graba $\times 6$, y elegir “ $\times 24$ ” graba $\times 3$. Los códigos 0 ($\times 1,5$) y 7 ($\times 192$, el valor por defecto del firmware) coinciden. **[VERIFICAR: comprobar qué código de ganancia estaba grabado durante los ensayos]**

9.5. Segunda célula de carga en el mismo bus

Se evaluó añadir una segunda célula con una Load Cell 4 Click (NAU7802) en el mismo bus I²C (programa `test_loadcell_dual_v2`). Surgieron dos problemas:

- **Convivencia en el bus.** Con la lectura en dos fases, el ZSC31014 no toleraba transacciones ajenas entre la petición y la recogida del dato: un reinicio de bus dirigido al NAU7802 estropeaba la siguiente lectura del ZSC31014. De ahí nació el árbitro de `i2c_bus.h`.
- **Ruido del NAU7802.** Las medidas sobre la placa, con ganancia 128 y en reposo, dieron unas 18.000 cuentas pico a pico con el regulador interno a 4,5 V (en caída, porque la placa se alimenta a 3,3 V); unas 3.500 con el regulador a 3,0 V y calibración interna de cero; y unas 350 al activar el *chopper* del amplificador, que equivalen a 0,49 μ V en la entrada y coinciden con las entradas cortocircuitadas. También se comprobó que esa Click no alimenta AVDD por fuera, así que el conversor debe usar el regulador interno; en caso contrario satura en $-8\,388\,608$.

La segunda célula se retiró del firmware final para simplificar el lazo y el bus. Los ensayos analizados usan sólo la célula del cable.

9.6. Otros problemas

Órdenes perdidas por los atajos de teclado

Un byte de CRC igual a una letra de atajo hacía que se perdiera la orden y se reiniciara el servo (sección 7). Se resolvió aceptando los atajos sólo con el decodificador en reposo.

Frecuencia de muestreo imposible

La interfaz llegó a mostrar 3813 Hz. Tras un reinicio del microcontrolador su reloj volvía a cero, la ventana de medida no se podaba nunca y se contaban todas las muestras acumuladas. Se detecta el salto de reloj y se reinicia la ventana.

Desplazamientos fantasma

Tras la programación de la célula, que bloquea el lazo cerca de un segundo, la integración multiplicaba ese hueco por la última velocidad. Se anula la base de tiempo tras cada operación bloqueante.

Órdenes que “no hacían nada”

Una acción mal escrita en la interfaz se ignoraba sin avisar. Se añadió una confirmación para toda orden, con el motivo del fallo cuando lo hay.

Revisión A de la placa

Conflictos de red en el bloque RS-485, entradas analógicas en pines sin ADC y amplificadores sin alimentar (tabla 5.1).

9.7. Lecciones aprendidas

- **Leer la hoja de datos entera**, no sólo las tablas de registros: la ventana del modo comando, la firma de la EEPROM, el umbral de POR y la codificación no lineal de la ganancia están en secciones distintas y todas han resultado críticas.
- **Medir antes de suponer**: la alimentación parásita sólo se descubrió con un polímetro, y el retardo de la ventana, con un barrido.
- **Cada operación debe responder**: las confirmaciones de órdenes y el modelo pregunta-respuesta de la célula han convertido fallos silenciosos en diagnósticos claros.
- **La sencillez da determinismo**: un lazo de periodo fijo con dueño único de cada bus ha dado mejor resultado que repartir el trabajo entre dos núcleos.
- **Conservar los datos brutos**: ha permitido recalibrar y reanalizar todos los ensayos para esta memoria sin repetirlos.

Capítulo 10

Resultados experimentales

Este capítulo presenta, primero, la caracterización del sistema de control y adquisición (temporización y repetibilidad) y, después, los resultados sobre el mecanismo: la evolución de la tensión del cable y del par del motor a lo largo del recorrido, la separación de sus componentes y la comparación con el modelo. Todas las figuras y cifras se obtienen con `memoria/figuras/generar_figuras.py` a partir de los ficheros de `cycles/`, sin ninguna edición manual.

10.1. Condiciones de ensayo

[COMPLETAR: fotografía del prototipo montado, con la caracola, el cable, la célula de carga, la barra, la masa y los finales de carrera identificados]

Los ensayos se realizaron los días 14 y 15 de septiembre de 2026 con la masa de 2 kg. Antes de cada serie se inicializó el accionamiento y se ejecutó el *homíng*. Cada ensayo es un ciclo automático que parte del origen (A), avanza a velocidad constante hasta el final de carrera B y vuelve al origen. Los parámetros del accionamiento fueron los de la tabla 7.4 (rigidez 10, rampas 5) [VERIFICAR: confirmar que no se modificaron durante la campaña].

Se grabaron 15 ficheros (tabla 10.1). Uno de ellos, a 300 rpm, se interrumpió a los 1,15 s y se descarta. Quedan 14 ensayos válidos con 2744 muestras: 12 a 30 rpm, que suman 14 ciclos porque dos de ellos tienen dos repeticiones, uno a 20 rpm y otro a 40 rpm.

Tabla 10.1: Ensayos grabados. Velocidad de consigna, repeticiones, muestras, duración y posición máxima alcanzada (final de carrera B).

Fichero	rpm	Rep.	Muestras	t (s)	$p_{\text{máx}}$ (rev)	Observaciones
20260914-183121	20	1	252	12,55	1,763	
20260914-184729	30	2	332	16,55	1,763	
20260914-190535	30	2	340	16,95	1,762	
20260914-190826	31	1	160	7,95	1,755	
20260914-194105	40	1	128	6,35	1,761	
20260914-194640	300	1	24	1,15	1,810	descartado
20260914-194648	30	1	176	8,75	1,717	inicio en $-0,10$
20260914-195242	30	1	168	8,35	1,755	
20260914-195540	30	1	172	8,55	1,760	
20260914-202156	30	1	168	8,35	1,752	
20260914-202425	30	1	168	8,36	1,758	
20260914-202801	30	1	168	8,35	1,750	

Fichero	rpm	Rep.	Muestras	t (s)	$p_{\text{máx}}$ (rev)	Observaciones
20260914-203733	30	1	172	8,55	1,756	
20260915-174420	30	1	168	8,35	1,782	
20260915-181346	30	1	172	8,55	1,759	ensayo tipo

Los ensayos se grabaron con una versión del servidor que todavía no escribía la calibración en la cabecera (`lc1_cuentas_por_N=0`). Como los ficheros contienen las cuentas brutas, los valores de fuerza de este capítulo se obtienen aplicando después la calibración provisional de la interfaz: 14,41 cuentas/N y 51,78 N en el punto de tara, con $F = \text{lc1_base}/14,41 + 51,78$ (sección 10.9).

10.2. Temporización del lazo de control

La figura 10.1 muestra la distribución del periodo entre muestras consecutivas y del tiempo de trabajo de cada vuelta del lazo, medidos por el propio firmware. En las 2744 muestras:

- el periodo medio es 50,004 ms, con una desviación típica de 0,066 ms. El 99,56 % de los periodos son exactamente de 50 ms y el resto de 51 ms, el grano de `millis()`. La frecuencia de muestreo es de 20,0 Hz y cumple RNF1 con holgura;
- el tiempo de trabajo del lazo está entre 10,82 y 11,02 ms, con media de 10,97 ms. Es el 22 % del periodo y deja 39 ms de margen. Su dispersión, de sólo 0,2 ms, confirma que el lazo está dominado por las tres transacciones Modbus, de duración muy regular (sección 6.2).

La latencia de parada (RNF2) está acotada por la estructura del lazo. Un final de carrera se lee y la consigna nula se escribe en la misma vuelta, así que la reacción es como máximo de un periodo más la rampa de deceleración del accionamiento. En el ensayo tipo, el final B se detecta en $p = 1,737$ rev y el eje se detiene en 1,759 rev: un sobrerrecorrido de 22 mrev, coherente con recorrer a 30 rpm durante un periodo (25 mrev).

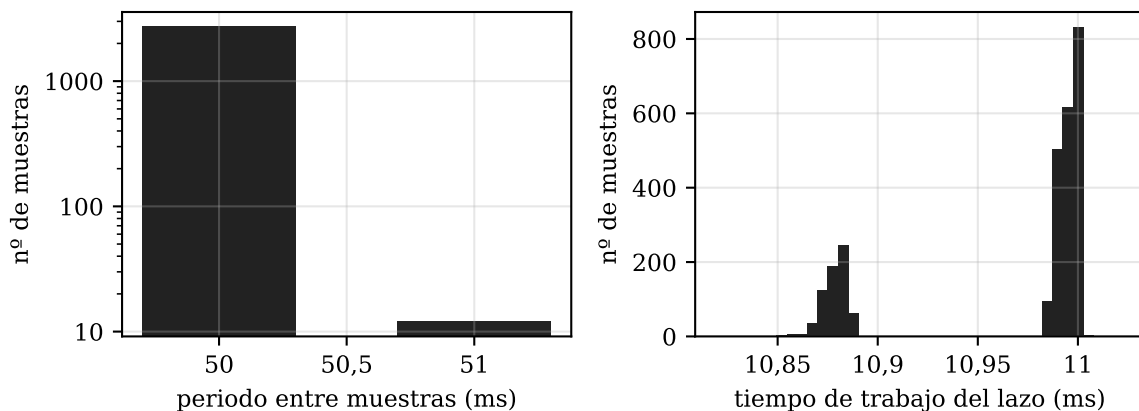


Figura 10.1: Izquierda: periodo entre muestras (escala logarítmica). Derecha: tiempo de trabajo por vuelta del lazo. Todos los ensayos válidos.

10.3. Ensayo tipo

La figura 10.2 muestra el ensayo del 15 de septiembre a las 18:13:46. La secuencia es la siguiente:

1. El eje arranca con el final A pulsado (sombreado rojo), que se libera a los 0,2s. La velocidad alcanza la consigna en unos 0,3s.
2. Hacia B, que corresponde a bajar la masa, la tensión del cable crece de forma continua. El par del motor se mantiene prácticamente constante hasta $p \approx 1,3$ rev y después disminuye. En ese último tramo la velocidad real cae a unas 20 rpm, pese a que la consigna sigue en 30 rpm.
3. A los 4,1s se activa el final B (sombreado azul). El firmware invierte la consigna y el eje vuelve hacia A, subiendo la masa, con un par mayor que a la ida por efecto de las pérdidas.
4. Al llegar a la posición $p \leq 0,02$ rev el ciclo termina. La posición final es $-9,5$ mrev y la duración total 8,55s.

La lectura bruta de la célula aparece en escalones de 200 ms porque sólo se transmite en la cabecera de cada lote. La lectura filtrada, que viaja en cada muestra, va retrasada respecto a ella por la media móvil de ocho lecturas.

10.4. Repetibilidad de posición

La figura 10.3 resume los 14 ciclos a 30 rpm.

- **Posición del final de carrera B:** media 1,7553 rev y desviación típica 13,4 mrev, con valores entre 1,7172 y 1,7815 rev. El valor más bajo corresponde al ensayo que arrancó a $-0,10$ rev; sin él, el rango se reduce a 1,750 rev a 1,782 rev. Esta dispersión combina la repetibilidad del propio final de carrera con la deriva de la posición integrada a lo largo de 1,75 revoluciones, así que es una cota superior de ambas: menos del 1 % del recorrido.
- **Posición final en A:** media 0,6 mrev, desviación típica 10,7 mrev y máximo absoluto 16,6 mrev. Siempre está dentro de la tolerancia de parada de ± 20 mrev. Esta cifra mide la precisión de la parada respecto a la posición integrada, no respecto al mecanismo, porque es la propia posición integrada la que decide la parada.

Para dar escala: con la reducción $N = 4$, 10 mrev de motor equivalen a $0,9^\circ$ de giro de la caracola.

10.5. Tensión del cable y par del motor frente a la posición

La figura 10.4 superpone todas las muestras de los ensayos a 30 rpm en las que el eje se mueve de forma estable, es decir, con velocidad del mismo signo que la consigna y al menos el 60 % de su valor. Se aprecian tres hechos:

1. Los ensayos son muy repetibles: las nubes de puntos de los distintos ciclos se superponen.

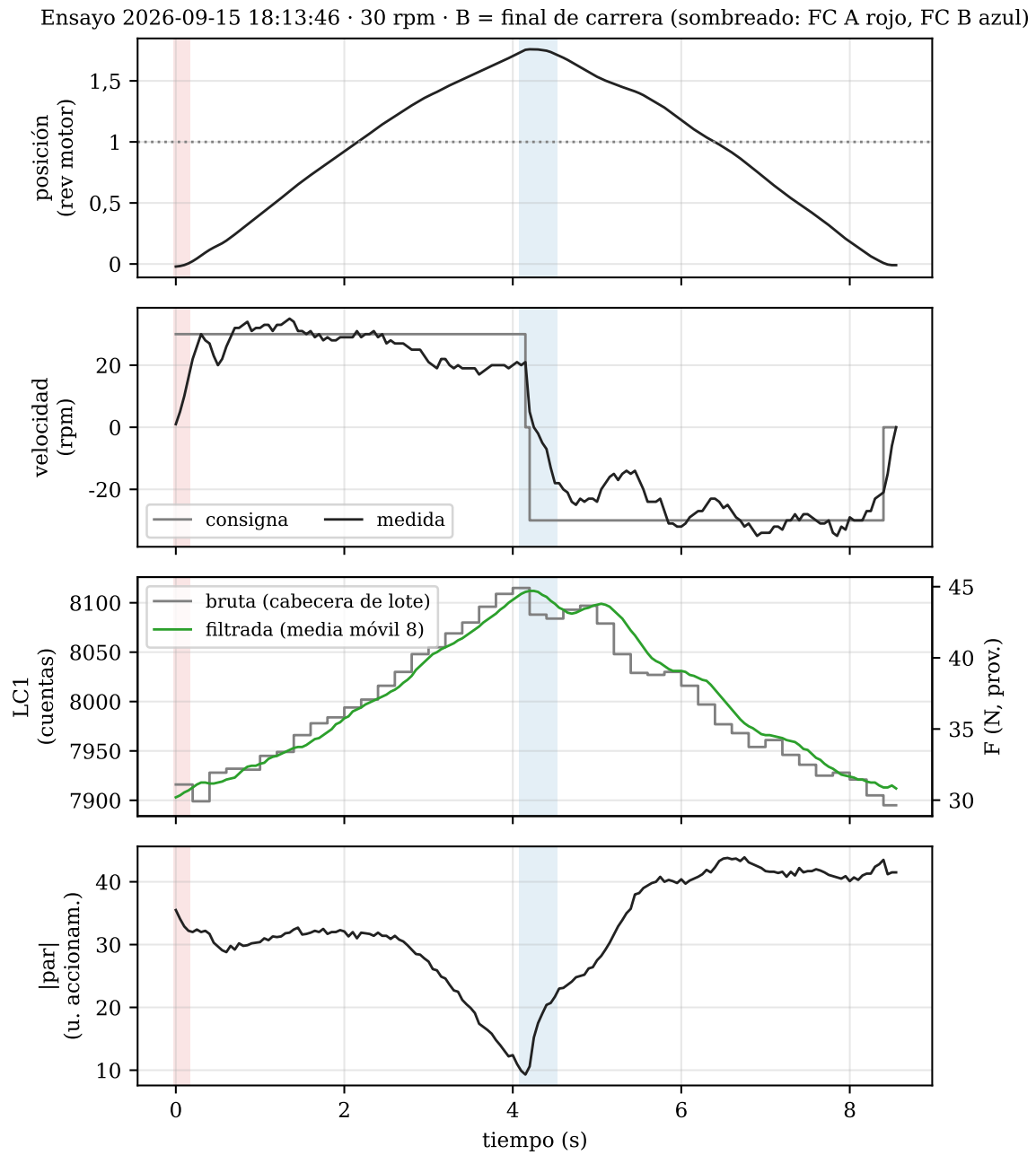


Figura 10.2: Ensayo tipo a 30 rpm: posición, velocidad, célula de carga (con la escala provisional en newtons a la derecha) y par del motor.

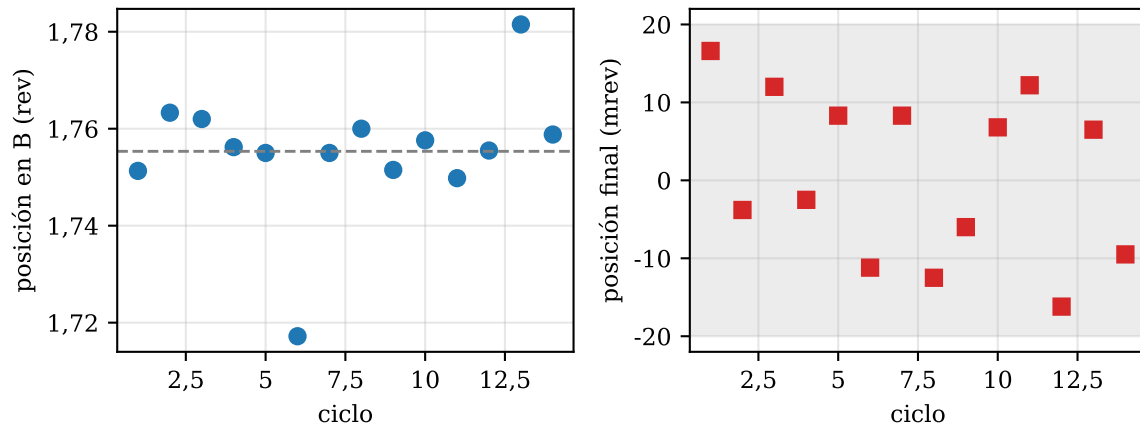


Figura 10.3: Repetibilidad en los 14 ciclos a 30 rpm. Izquierda: posición a la que se alcanza B. Derecha: posición final en A, con la banda de tolerancia de ± 20 mrev.

2. La tensión crece de forma monótona desde el origen hasta B, lo que confirma que ir hacia B baja la masa (el cable se alarga y, por la ecuación (3.5), la tensión aumenta).
3. El par del motor es mayor hacia A (subida) que hacia B (bajada) en todo el recorrido, como corresponde a unas pérdidas que se oponen siempre al movimiento. Su valor es casi constante hasta 1,3 rev y cae en el tramo final.

10.6. Separación de componentes y efecto de la caracola

Para aplicar el método de la sección 3.5, las muestras se agrupan en intervalos de posición de 0,05 rev y en cada uno se promedian por separado la fuerza y el par de cada sentido. Sólo se usan los intervalos con al menos ocho muestras en ambos sentidos, que resultan 34 entre 0,03 y 1,73 rev. La figura 10.5 muestra el resultado.

Tensión. Promediando ambos sentidos, la tensión pasa de 30,6 N en el origen a 42,8 N en B. Las curvas de ida y vuelta difieren en promedio sólo 0,27 N: una vez agrupadas por posición, la histéresis aparente que introduce el filtro de la célula prácticamente desaparece.

Pérdidas. La componente de pérdidas es casi constante a lo largo del recorrido, con un valor medio de 4,9 u.a., el 14 % de la componente gravitatoria. Una pérdida independiente de la posición y del sentido corresponde a un rozamiento de tipo Coulomb en la transmisión y en el guiado del cable.

Compensación en el tramo central. Entre 0,3 y 1,2 rev, lejos de las aceleraciones del arranque y del final de carrera:

Magnitud	Mínimo	Máximo	Variación relativa
Tensión del cable F	31,9 N	37,8 N	17,0 %
Par gravitatorio del motor τ_g	36,4 u.a.	38,9 u.a.	6,4 %

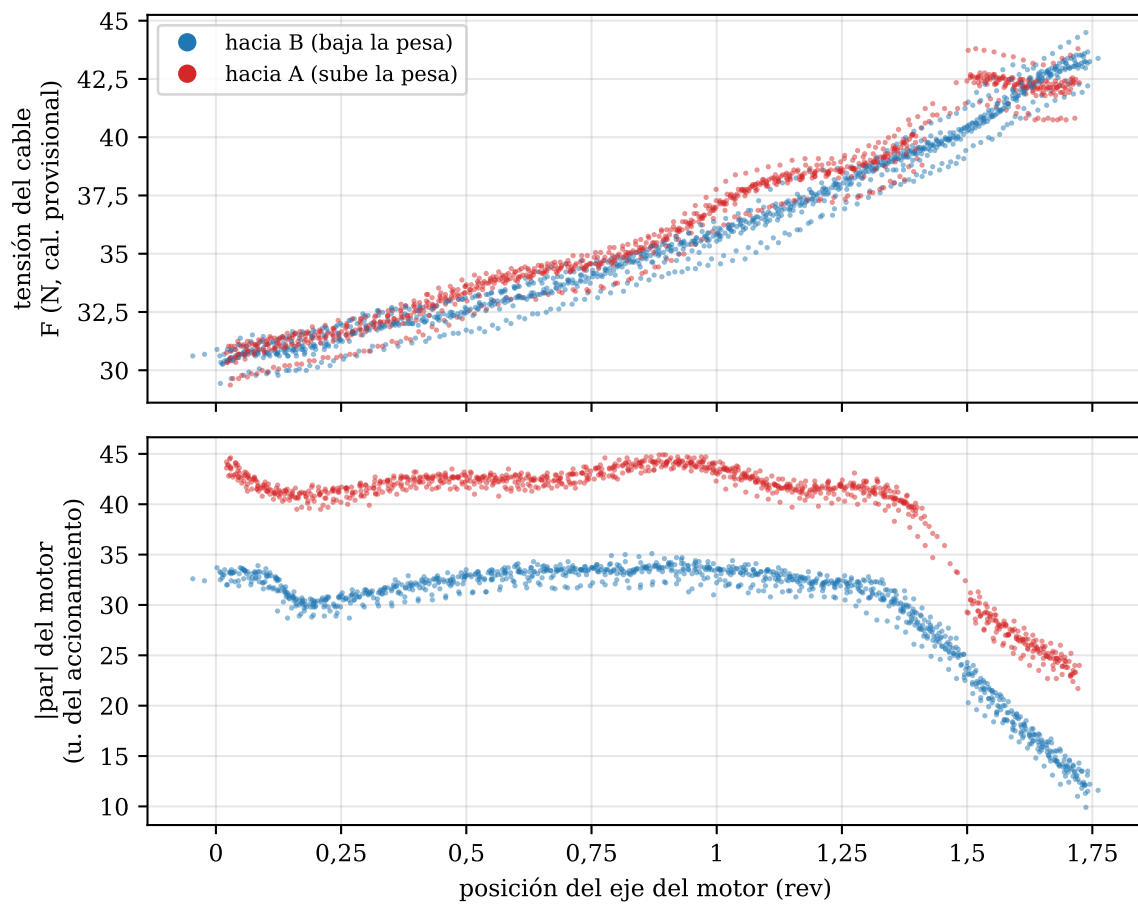


Figura 10.4: Tensión del cable (arriba) y par del motor (abajo) frente a la posición, en los 12 ensayos a 30 rpm. Azul: hacia B (baja la masa). Rojo: hacia A (sube la masa).

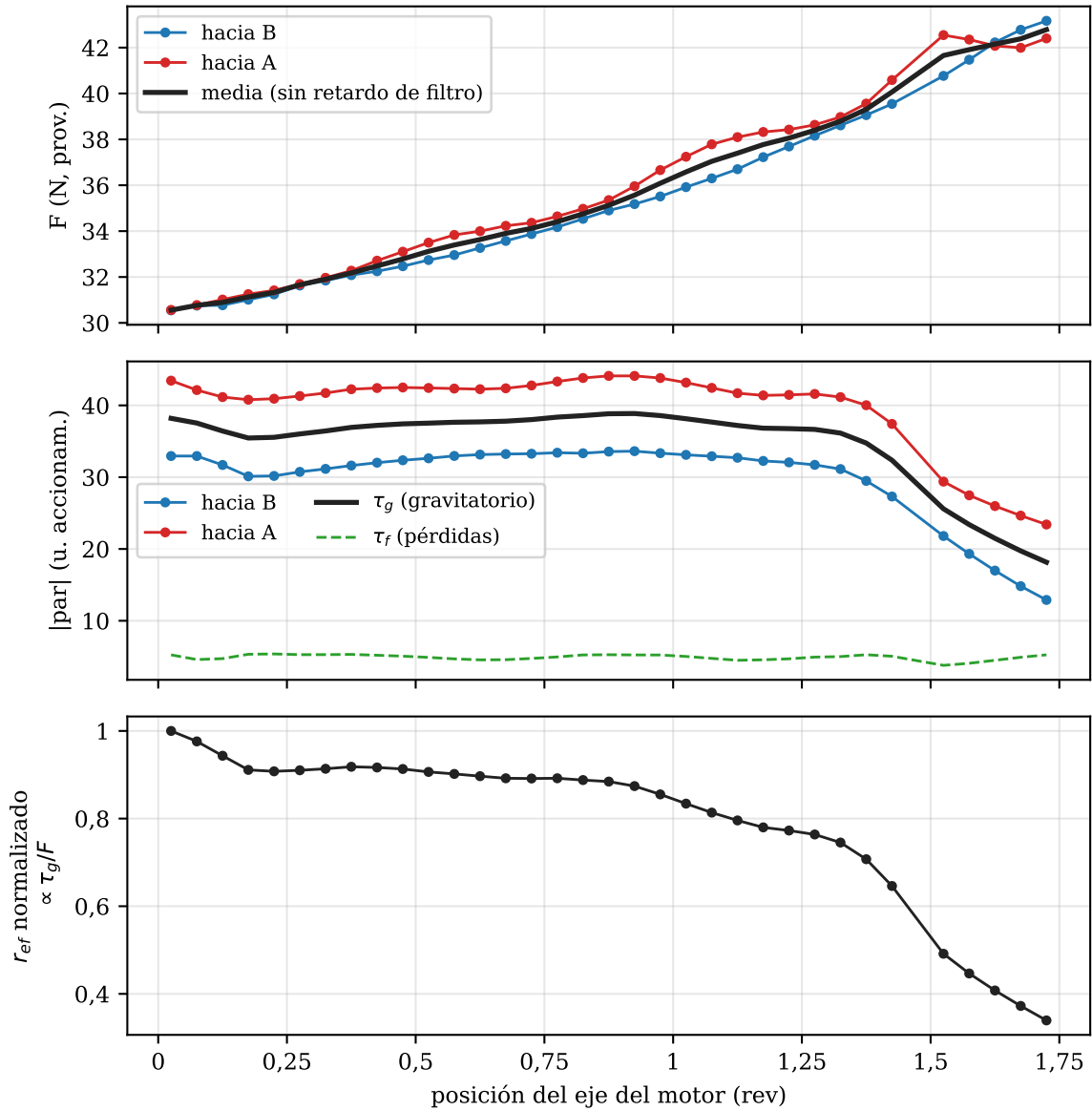


Figura 10.5: Arriba: tensión media por intervalo en cada sentido y su promedio. Centro: par en cada sentido, componente gravitatoria τ_g y de pérdidas τ_f . Abajo: radio efectivo relativo τ_g/F , normalizado a su máximo.

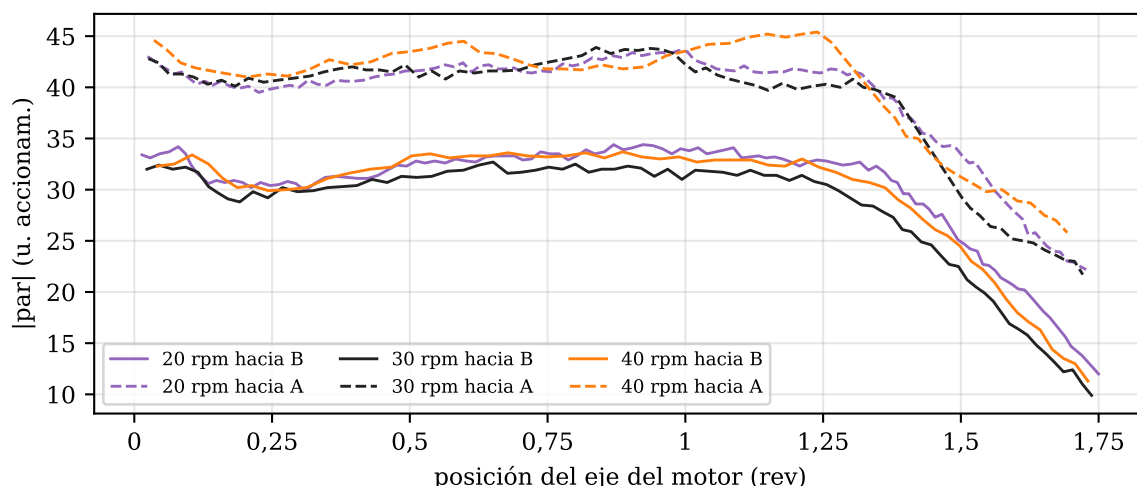


Figura 10.6: Par del motor frente a la posición a 20, 30 y 40 rpm. Línea continua: hacia B. Discontinua: hacia A.

Con un tambor cilíndrico, el par sería proporcional a la tensión (ecuación (3.7)) y variaría también un 17 %. La caracola reduce esa variación a un 6,4 %: compensa del orden del 60 % de la variación de la carga. El resultado sólo usa variaciones relativas, así que no depende de la escala desconocida del par, y tampoco de la calibración absoluta de la célula, salvo en el valor de la fuerza en la tara, que fija el denominador. **Este es el resultado experimental central del trabajo.**

Tramo final. A partir de 1,3 rev el par gravitatorio cae hasta 18 u.a. mientras la tensión sigue creciendo, y el radio efectivo baja al 34 % de su valor inicial. Hay tres explicaciones posibles, que no se han podido discriminar con estos ensayos: (a) el perfil real de la caracola se estrecha bruscamente al final de la espiral; (b) el cable llega al extremo de la espiral y cambia su punto de salida; (c) la caída de velocidad observada en ese tramo (figura 10.2) rompe la hipótesis de régimen cuasiestático e igual velocidad en ambos sentidos [**VERIFICAR: medir el perfil de radio de la caracola real y comprobar la salida del cable al final del recorrido**].

10.7. Influencia de la velocidad

La figura 10.6 compara el par en ensayos a 20, 30 y 40 rpm. Los pares medios son 38,7 u.a., 38,4 u.a. y 39,7 u.a. hacia A y 30,2 u.a., 27,6 u.a. y 28,9 u.a. hacia B, respectivamente. No se aprecia una tendencia con la velocidad: las diferencias, inferiores al 10 %, están en el orden de la variación entre ensayos. Esto confirma que en este rango las pérdidas no son viscosas y que el ensayo es cuasiestático. La posición de B coincide en los tres casos con una dispersión de 4 mrev (1,763 rev, 1,759 rev y 1,761 rev).

10.8. Coherencia con el modelo

Aunque el modelo se ha evaluado con cotas nominales, conviene comprobar que los órdenes de magnitud son coherentes:

- La tensión medida (30,6 N a 42,8 N) está dentro del rango teórico con la masa de 2 kg (28,5 N a 56,2 N).
- Invertiendo $T = Kc$, esa tensión corresponde a un cable de 209,6 mm a 293,2 mm y, por la ecuación (3.2), a una barra que pasa de unos 63° a unos 7°.
- Esos 83,6 mm de cable se pagan en 1,755 rev de motor, es decir, 0,439 rev de caracola con $N = 4$. El radio medio de enrollamiento resulta de 30,3 mm, un valor razonable para la pieza.
- Con ese radio medio y $F \approx 35$ N, el par gravitatorio en el motor sería de unos 0,27 N m (con $\eta = 1$). Comparado con $\tau_g \approx 37,5$ u.a., cada unidad del accionamiento equivaldría a unos 7 mN m. Es sólo una estimación de orden de magnitud; sirve para contrastar con el manual del accionamiento [VERIFICAR: escala del registro 16387 y par nominal del motor].

10.9. Limitaciones de las medidas

- **Calibración de la célula.** La constante de 14,41 cuentas/N y el valor de 51,78 N en la tara son provisionales: no proceden de una calibración con masas patrón. Las variaciones relativas del tramo central dependen poco de ellos, pero los valores absolutos en newtons deben tomarse como orientativos. Se propone el procedimiento de calibración del apartado siguiente.
- **Filtro de la célula.** La media móvil de ocho lecturas válidas introduce un retardo de varias décimas de segundo. Promediar ambos sentidos lo compensa en primera aproximación, pero limita el análisis de transitorios.
- **Lectura bruta por lote.** La lectura sin filtrar sólo se transmite una vez cada 200 ms.
- **Posición integrada.** No se ha contrastado con una medida independiente (codificador del accionamiento o medida angular de la barra).
- **Unidades del par.** El par se usa en unidades internas del accionamiento.
- **Una sola masa y un solo prototipo.** No se ha estudiado la dependencia con la carga ni con el montaje.

10.9.1. Procedimiento de calibración propuesto

1. Colgar la célula en vertical con su cadena de montaje y tarar sin carga.
2. Aplicar masas patrón en escalones crecientes y decrecientes que cubran el rango de uso (por ejemplo, de 0 a 6 kg en pasos de 0,5 kg), registrando 10 s por escalón con el eje parado.
3. Ajustar por mínimos cuadrados la recta cuentas-fuerza y obtener la sensibilidad, el cero, el error de linealidad, la histéresis entre carga y descarga y la repetibilidad en tres series.
4. Estimar la incertidumbre combinada, con las contribuciones de las masas patrón, la resolución, el ruido, la linealidad y la histéresis, y expresarla con $k = 2$.

5. Introducir la nueva constante en la interfaz y recalcular los ensayos existentes, cuyas cuentas brutas se conservan.

Capítulo 11

Planificación, presupuesto e impacto

11.1. Planificación

La planificación real se ha reconstruido a partir del historial del repositorio git: 35 *commits* entre el 4 de marzo y el 14 de septiembre de 2026, concentrados en ocho semanas (figura 11.1). El historial sólo refleja el trabajo sobre código y diseño electrónico. Los periodos sin *commits* corresponden a otras actividades (montaje mecánico, pruebas sin cambios de código, periodo lectivo) **[COMPLETAR: describir las actividades de abril a junio y de julio a agosto]**.

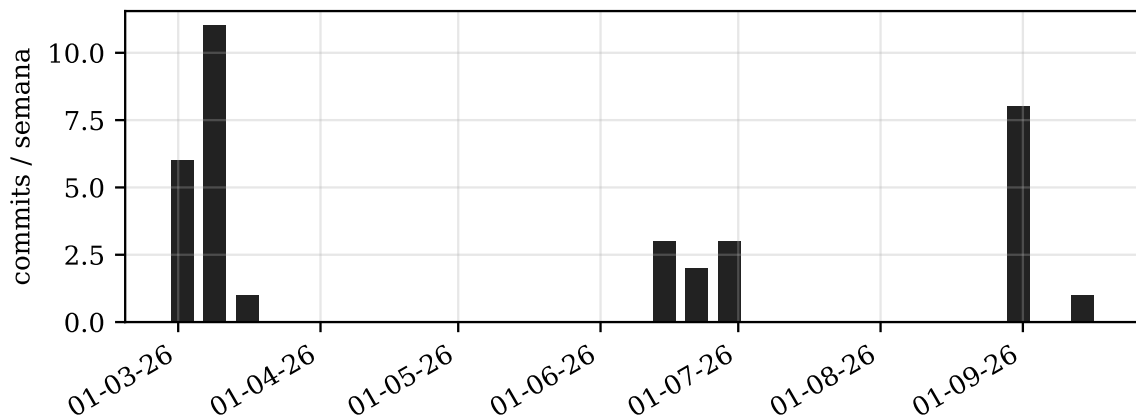


Figura 11.1: *Commits* por semana en el repositorio del proyecto.

11.1.1. Fases

F1. Exploración del accionamiento (2–10 de marzo)

Primeras pruebas de comunicación Modbus con una placa XIAO ESP32-S3: escaneo de direcciones, modos par y velocidad, monitor de terminal en Python, parámetros del accionamiento y resistencia de frenado. Culmina en un firmware v1.0 con un regulador de fuerza en modo par.

F2. Diseño de la placa adaptadora (12–17 de marzo)

Esquema y colocación de la revisión A, con 18 copias de seguridad automáticas en seis días.

F3. Migración a Raspberry Pi Pico 2 y validación de la placa (20–30 de junio)

Programas de prueba de entradas y salidas, célula de carga simple y doble, RS-485 (bucle

local, transmisión y recepción) y escaneo Modbus. Versión de doble núcleo.

F4. Firmware de posición en modo velocidad (4 de julio)

Movimiento A→B con integración de la posición y monitor asociado.

F5. Firmware v2 y supervisión web (2–6 de septiembre)

Homing con dos finales de carrera, servidor FastAPI, página de supervisión, segunda célula con ganancia, ciclos automáticos y grabación de ensayos.

F6. Campaña de ensayos, análisis y memoria (14–21 de septiembre)

Quince ensayos, visor de ensayos con modelo teórico, análisis de datos y redacción.

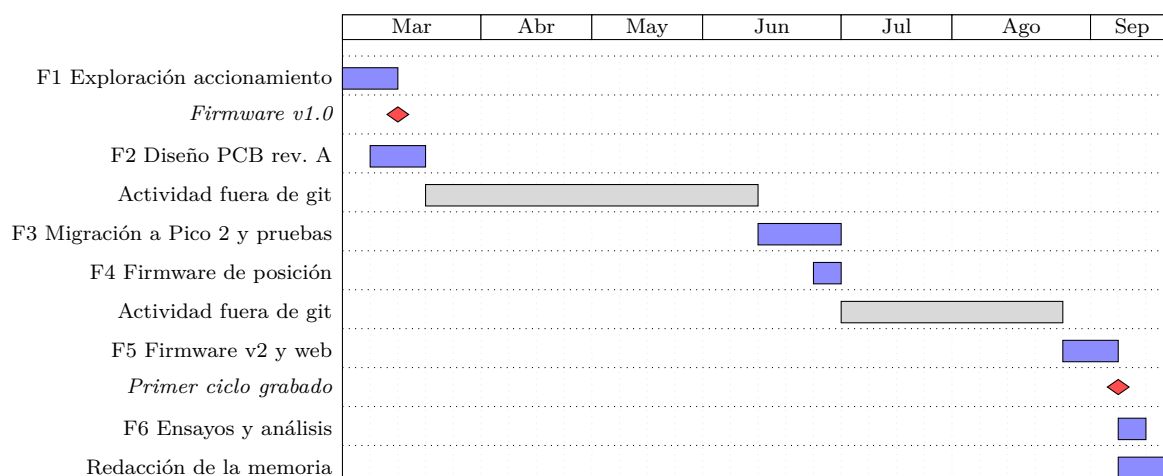


Figura 11.2: Diagrama de Gantt reconstruido (semanas desde el 2 de marzo de 2026).

11.2. Presupuesto

11.2.1. Recursos humanos

La dedicación de un TFG se fija en créditos ECTS, cada uno de 25 a 30 horas de trabajo [18]. La tabla 11.1 reparte una dedicación estimada por fases **[VERIFICAR: ajustar las horas y los créditos del TFG en la titulación]**. Para valorar el trabajo se toma como supuesto un coste de 30 €/h para un ingeniero o ingeniera junior y de 45 €/h para la tutoría.

Tabla 11.1: Dedicación estimada y coste de personal.

Actividad	Horas	€/h	Coste (€)
Estudio y documentación	30	30	900
Exploración del accionamiento (F1)	35	30	1.050
Diseño de la placa (F2)	45	30	1.350
Pruebas de placa y comunicaciones (F3)	45	30	1.350
Firmware (F4–F5)	70	30	2.100
Software de supervisión (F5)	50	30	1.500
Ensayos y análisis (F6)	25	30	750
Redacción de la memoria	50	30	1.500
Subtotal trabajo técnico	350		10.500
Tutoría	25	45	1.125
Total recursos humanos			11.625

11.2.2. Material

Tabla 11.2: Material del sistema.

Elemento	Cant.	Precio unit. (€)	Total (€)
Raspberry Pi Pico 2	1	[COMPLETAR:]	
Transceptor RS-485 THVD1400	1	[COMPLETAR:]	
Interruptor de lado alto TPS4H000-Q1	1	[COMPLETAR:]	
Amplificador operacional MCP6294	1	[COMPLETAR:]	
Diodos zéner BZX384	4	[COMPLETAR:]	
Resistencias y condensadores SMD 0805	25	[COMPLETAR:]	
Bornas de tornillo y tiras de pines	–	[COMPLETAR:]	
Fabricación de la PCB	1	[COMPLETAR:]	
Load Cell 2 Click (ZSC31014)	1	[COMPLETAR:]	
Célula de carga	1	[COMPLETAR:]	
Servomotor paso a paso de lazo cerrado con accionamiento	1	[COMPLETAR:]	
Fuente de alimentación del accionamiento	1	[COMPLETAR:]	
Finales de carrera	2	[COMPLETAR:]	
Caracola, barra, cable, estructura y masa de 2 kg	–	[COMPLETAR:]	
Total material			[COMPLETAR:]

11.2.3. Equipos y software

Todo el software empleado es libre o gratuito (KiCad, Arduino IDE con el núcleo Arduino-Pico, Python, FastAPI, Chart.js, git, $\text{\LaTeX}$), por lo que su coste es nulo. Los equipos se imputan por amortización lineal,

$$A = C \frac{t_{\text{uso}}}{t_{\text{vida}}}, \quad (11.1)$$

con C el coste de adquisición, t_{uso} el tiempo de uso en el proyecto y t_{vida} la vida útil (tabla 11.3).

Tabla 11.3: Amortización de equipos.

Equipo	Coste (€)	Vida útil (meses)	Uso (meses)	Amortización (€)
Ordenador portátil	[COMPLETAR:]	48	7	
Polímetro	[COMPLETAR:]	60	7	
Fuente de laboratorio	[COMPLETAR:]	60	7	
Total	[COMPLETAR:]			

11.2.4. Resumen

Tabla 11.4: Presupuesto total.

Concepto	Importe (€)
Recursos humanos	11.625
Material	[COMPLETAR:]
Amortización de equipos	[COMPLETAR:]
Subtotal	[COMPLETAR:]
Costes indirectos (10 %, supuesto)	[COMPLETAR:]
Base imponible	[COMPLETAR:]
IVA (21 %)	[COMPLETAR:]
Total	[COMPLETAR:]

11.3. Impacto

11.3.1. Impacto técnico y económico

El resultado del capítulo 3 tiene una consecuencia directa en el dimensionado: para un recorrido dado, un perfil de par constante reduce el par de pico que debe dar el motor. Con la geometría nominal la reducción es del 24,7 %, lo que puede permitir un motor y un accionamiento de un tamaño inferior, con menor coste y menor consumo de materiales. Además, a velocidad constante las pérdidas por efecto Joule en el motor son proporcionales a $\int \tau^2 d\varphi$. Para un trabajo fijo, esa integral es mínima cuando el par es constante (desigualdad de Cauchy-Schwarz). Con la geometría nominal, el perfil ideal reduce esas pérdidas un 3,5 % respecto al tambor cilíndrico.

La plataforma desarrollada es además reutilizable. La placa adaptadora, el firmware y la aplicación web forman un banco genérico para cualquier eje accionado por Modbus con medida de fuerza, y su coste de material es muy inferior al de un sistema de adquisición comercial.

11.3.2. Objetivos de Desarrollo Sostenible

Tabla 11.5: Relación del trabajo con los Objetivos de Desarrollo Sostenible [19].

ODS	Contribución
4. Educación de calidad	Banco de ensayo abierto y documentado, útil como práctica de laboratorio de mecanismos, accionamientos y adquisición de datos.
7. Energía asequible y no contaminante	Reducción de las pérdidas en el motor mediante un perfil de par uniforme.
9. Industria, innovación e infraestructura	Integración de tecnologías industriales (Modbus, RS-485) con herramientas libres de bajo coste.
12. Producción y consumo responsables	Dimensionado de actuadores más pequeños; módulo microcontrolador en zócalo, reutilizable; datos brutos conservados para no repetir ensayos.

11.3.3. Aspectos éticos, legales y de seguridad

El prototipo es un equipo de laboratorio que no se comercializa. Aun así, su evolución hacia una máquina de uso por terceros tendría que considerar la Directiva 2006/42/CE de máquinas, que el Reglamento (UE) 2023/1230 sustituye a partir de enero de 2027, la Directiva 2014/30/UE de compatibilidad electromagnética y la Directiva 2011/65/UE sobre sustancias peligrosas (RoHS). La sección 4.5 discute las limitaciones de seguridad del diseño actual frente a ISO 13849-1 [16].

En cuanto a la ética profesional, se ha priorizado la trazabilidad de los datos: se conservan las medidas brutas, las figuras se generan con un programa reproducible y las limitaciones y los datos provisionales se señalan de forma explícita en la memoria.

Capítulo 12

Conclusiones y trabajo futuro

12.1. Grado de cumplimiento de los objetivos

Obj.	Resultado	Estado
O1	Modelo analítico con tensión proporcional a la longitud de cable, perfil ideal de par constante en forma cerrada, demostración de que minimiza el par de pico y comparación con el cilindro y la espiral lineal (capítulo 3).	Cumplido
O2	Placa adaptadora con RS-485, entradas digitales y analógicas, salidas de potencia y dos zócalos mikroBUS. Revisión A analizada y corregida en la revisión B (capítulo 5).	Cumplido; falta versionar la revisión B
O3	Firmware determinista a 20 Hz con Modbus, célula de carga, <i>homing</i> , ciclos y protecciones (capítulo 6).	Cumplido
O4	Protocolo binario con CRC-8, telemetría por lotes y modelo pregunta-respuesta (capítulo 7).	Cumplido
O5	Servidor y páginas de supervisión y de análisis con modelo teórico y grabación automática (capítulo 8).	Cumplido
O6	Caracterización temporal y de repetibilidad; cuantificación de la compensación de par (capítulo 10).	Cumplido, con calibración provisional
O7	Documentación de problemas, planificación, presupuesto e impacto (capítulos 9 y 11).	Cumplido

12.2. Conclusiones

1. **La tensión del cable es proporcional a su longitud libre**, $T = mg(L_3 + L_4) c / (L_2 L_3)$, con independencia del ángulo de la barra. Este resultado simplifica todo el análisis posterior.
2. **El perfil de par constante es el óptimo** en cuanto a par de pico, y existe en forma cerrada: $r(\varphi) = C / \sqrt{c_0^2 + 2C\varphi}$. La reducción de par de pico frente al tambor cilíndrico sólo depende de la geometría, $1 - (c_{\text{máx}} + c_{\text{mín}}) / (2c_{\text{máx}})$, y vale un 24,7 % con las cotas nominales. Una espiral lineal bien elegida consigue el 85 % de esa mejora.

3. **La caracola compensa la variación de carga en el prototipo real.** En el tramo central del recorrido la tensión del cable varía un 17 % y el par gravitatorio del motor sólo un 6,4 %. El método de separar el par en componentes gravitatoria y de pérdidas a partir de recorridos en ambos sentidos permite obtener esta conclusión sin conocer la escala del par del accionamiento.
4. **Las pérdidas del mecanismo son de tipo Coulomb:** aproximadamente constantes a lo largo del recorrido (14 % del par gravitatorio) e independientes de la velocidad entre 20 y 40 rpm.
5. **La plataforma cumple los requisitos de temporización:** periodo de 50,004 ms con dispersión de 0,066 ms y un tiempo de trabajo del 22 % del periodo. Llegar a este determinismo exigió abandonar la versión de doble núcleo en favor de un único dueño de cada bus en un lazo de periodo fijo.
6. **Los componentes con reglas estrictas requieren diagnóstico.** El acondicionador ZSC31014 sólo pudo configurarse de forma fiable después de descubrir y eliminar la alimentación parásita a través de las líneas I²C, y de medir la ventana del modo comando con un barrido de retardos en lugar de copiar valores de otros diseños.
7. **Validar cada subsistema por separado** con programas de prueba específicos permitió aislar rápidamente los fallos de hardware (revisión A de la placa) y de comunicación.

12.3. Trabajo futuro

12.3.1. Medida y validación

- Calibrar la célula de carga con masas patrón y estimar su incertidumbre (sección 10.9).
- Confirmar la escala del registro de par del accionamiento y expresar los pares en N m.
- Medir el perfil real de la caracola y las cotas del mecanismo, regenerar el modelo y explicar la caída de par del tramo final.
- Leer la posición del codificador del accionamiento por Modbus en lugar de integrar la velocidad.
- Ensayar varias masas y fabricar, por ejemplo por impresión 3D, un tambor cilíndrico y otro con el perfil ideal (3.11), para comparar los tres perfiles en el mismo banco.

12.3.2. Firmware y comunicaciones

- Leer velocidad y par en una sola transacción Modbus (función 0x03 sobre los registros 16385–16387). Eliminaría una de las tres transacciones por periodo y ahorraría unos 3 ms.
- Enviar en cada muestra la posición del firmware y la lectura sin filtrar de la célula, y sincronizar la lectura con la señal INT del ZSC31014.
- Corregir la codificación de los bits de estado del ZSC31014 según la hoja de datos (sección 9.4).

- Incluir el modo del ciclo en el paquete 0x05 para que la cabecera del CSV sea correcta.
- Respetar el silencio de 1,75 ms recomendado por Modbus por encima de 19200 baudios.

12.3.3. Hardware y seguridad

- Versionar y fabricar la revisión B de la placa, con los ajustes de resistencias propuestos.
- Cablear los finales de carrera y una seta de emergencia a la entrada STO del accionamiento, de modo que la parada no dependa del software.
- Hacer que el servidor escuche sólo en la máquina local o exija autenticación, y añadir pruebas automáticas del protocolo.

12.4. Valoración personal

[**COMPLETAR:** Valoración personal: qué se ha aprendido, qué dificultades han sido más formativas y qué se haría de otra manera.]

Bibliografía

- [1] D. S. Landes, *Revolution in Time: Clocks and the Making of the Modern World*. Cambridge, MA: Harvard University Press, 1983.
- [2] J. L. Herder, *Energy-free systems: theory, conception and design of statically balanced spring mechanisms*, Tesis doctoral, Delft University of Technology, 2001.
- [3] S. Seriani y P. Gallina, “Variable radius drum mechanisms,” *Journal of Mechanisms and Robotics*, vol. 8, n.º 2, art. 021016, 2016.
- [4] J. Bieber, D. Bernstein y M. Beitelschmidt, “Variable radius drum design for cable-driven parallel robots based on maximum load profile,” en *Cable-Driven Parallel Robots (CableCon 2023)*, Mechanisms and Machine Science, vol. 132. Cham: Springer, 2023. doi: 10.1007/978-3-031-32322-5_22.
- [5] T. Shen y K. Yano, “An innovative spiral pulley that optimizes cable tension variation for superior balancing performance,” *Journal of Robotics and Mechatronics*, vol. 34, n.º 3, pp. 599–606, 2022. doi: 10.20965/jrm.2022.p0599.
- [6] G. Endo, H. Yamada, A. Yajima, M. Ogata y S. Hirose, “A passive weight compensation mechanism with a non-circular pulley and a spring,” en *Proc. IEEE International Conference on Robotics and Automation (ICRA)*, 2010.
- [7] T. Kenjo y A. Sugawara, *Stepping Motors and Their Microprocessor Controls*, 2.^a ed. Oxford: Oxford University Press, 1994.
- [8] R. Pallàs-Areny y J. G. Webster, *Sensors and Signal Conditioning*, 2.^a ed. Nueva York: Wiley, 2001.
- [9] K. Hoffmann, *An Introduction to Stress Analysis and Transducer Design using Strain Gauges*. Darmstadt: HBM, 2012.
- [10] Modbus Organization, *MODBUS Application Protocol Specification V1.1b3*, 2012.
- [11] Modbus Organization, *MODBUS over Serial Line: Specification and Implementation Guide V1.02*, 2006.
- [12] Telecommunications Industry Association, *TIA/EIA-485-A: Electrical Characteristics of Generators and Receivers for Use in Balanced Digital Multipoint Systems*, 1998.
- [13] NXP Semiconductors, *UM10204: I²C-bus specification and user manual*, rev. 7.0, 2021.
- [14] MikroElektronika, *mikroBUS Standard Specification*, v2.00.
- [15] I. Fette y A. Melnikov, *The WebSocket Protocol*, RFC 6455, IETF, 2011.
- [16] ISO, *ISO 13849-1: Safety of machinery — Safety-related parts of control systems — Part 1: General principles for design*, 2023.
- [17] IEC, *IEC 61800-5-2: Adjustable speed electrical power drive systems — Part 5-2: Safety requirements — Functional*, 2016.

- [18] Real Decreto 1125/2003, de 5 de septiembre, por el que se establece el sistema europeo de créditos y el sistema de calificaciones en las titulaciones universitarias. BOE n.º 224, 2003.
- [19] Naciones Unidas, *Transformar nuestro mundo: la Agenda 2030 para el Desarrollo Sostenible*, Resolución A/RES/70/1, 2015.
- [20] Raspberry Pi Ltd, *RP2350 Datasheet* y *Raspberry Pi Pico 2 product brief*. Disponible en: <https://www.raspberrypi.com/products/raspberry-pi-pico-2/>.
- [21] Renesas Electronics, *ZSC31014 RBicLite Digital Output Sensor Signal Conditioner — Datasheet*. Disponible en: <https://www.renesas.com/en/products/zsc31014>.
- [22] Nuvoton Technology, *NAU7802: 24-bit dual-channel ADC for bridge sensors — Datasheet*.
- [23] Texas Instruments, *THVD1400: 3.3-V to 5-V RS-485 transceiver with IEC ESD protection — Datasheet*. Disponible en: <https://www.ti.com/product/THVD1400>.
- [24] Texas Instruments, *TPS4H000-Q1: 40-V, 1000-mΩ 4-channel smart high-side switch — Datasheet*. Disponible en: <https://www.ti.com/product/TPS4H000-Q1>.
- [25] Microchip Technology, *MCP6291/1R/2/3/4/5: 1.0 mA, 10 MHz rail-to-rail op amps — Datasheet*.
- [26] Nexperia, *BZX384 series: Voltage regulator diodes — Datasheet*.
- [27] MikroElektronika, *Load Cell 2 Click — Documentación del producto*. Disponible en: <https://www.mikroe.com>.
- [28] E. F. Philhower III *et al.*, *Arduino-Pico: Raspberry Pi Pico Arduino core for RP2040 and RP2350 boards*. Disponible en: <https://github.com/earlephilhower/arduino-pico>.
- [29] D. Walker, *ModbusMaster: Enlighten your Arduino to be a Modbus master*. Disponible en: <https://github.com/4-20ma/ModbusMaster>.
- [30] S. Ramírez, *FastAPI*. Disponible en: <https://fastapi.tiangolo.com>.
- [31] Encode OSS, *Uvicorn: an ASGI web server*. Disponible en: <https://www.uvicorn.org>.
- [32] C. Liechti, *pySerial*. Disponible en: <https://github.com/pyserial/pyserial>.
- [33] Chart.js contributors, *Chart.js*. Disponible en: <https://www.chartjs.org>.
- [34] KiCad Project, *KiCad EDA 9.0*. Disponible en: <https://www.kicad.org>.
- [35] J. D. Hunter, “Matplotlib: A 2D graphics environment,” *Computing in Science & Engineering*, vol. 9, n.º 3, pp. 90–95, 2007.

Apéndice A

Asignación de pines

A.1. Raspberry Pi Pico 2 (revisión B de la placa)

Tabla A.1: Asignación de GPIO según el esquema de la revisión B y el firmware.

GPIO	Pin	Señal	Función
GP0	1	INT1	Interrupción de la célula 1 (MD1)
GP1	2	EN1	Habilitación / reinicio de las placas Click (LC1_EN_GPIO)
GP2	4	SDA1	I ² C1 SDA (LC_SDA_PIN)
GP3	5	SCL1	I ² C1 SCL (LC_SCL_PIN)
GP4	6	INT2	Interrupción de la célula 2 (MD2)
GP5, GP6	7, 9	–	Libres
GP7	10	SIN4	Entrada del canal 4 del TPS4H000
GP8	11	SIN3	Entrada del canal 3
GP9	12	SIN2	Entrada del canal 2
GP10	14	SIN1	Entrada del canal 1
GP11	15	DST	Fallo global del TPS4H000 (drenador abierto, <i>pull-up</i>)
GP12	16	DE	Dirección del transceptor RS-485 (MB_EN_GPIO)
GP13	17	RO	Recepción RS-485, UART PIO (MB_RX_GPIO)
GP14	19	DI	Transmisión RS-485, UART PIO (MB_TX_GPIO)
GP15	20	DIAG_EN	Habilitación del diagnóstico del TPS4H000
GP16	21	DIN1	Final de carrera A (PIN_LIMIT_A)
GP17	22	DIN2	Final de carrera B (PIN_LIMIT_B)
GP18	24	DIN3	Entrada digital libre
GP19	25	DIN4	Entrada digital libre
GP20	26	COMP	Salida del comparador (umbral 2,5 V)
GP21, GP22	27, 29	–	Libres
GP26	31	AOUT3	ADC0: entrada analógica AIN3
GP27	32	AOUT1	ADC1: entrada analógica AIN1

GPIO	Pin	Señal	Función
GP28	34	AOUT2	ADC2: entrada analógica AIN2
VBUS	40	+5V	Alimentación de los amplificadores
3V3	36	+3.3V	Transceptor RS-485 y placas Click

Nota. El firmware define `LC1_INT_GPIO = 4`, mientras que en el esquema de la revisión B la interrupción de la célula 1 llega a GP0 y GP4 corresponde a la célula 2. El firmware sólo usa ese pin para llevarlo a nivel bajo durante el apagado de la célula (sección 9.2) **[VERIFICAR: comprobar a qué GPIO está conectado INT en la placa montada]**.

A.2. Bornas de campo

Tabla A.2: Bornas de la placa (revisión B).

Borna	Pin	Señal
J5 (RS-485)	1 / 2 / 3	A / B / GND
Terminal de salidas	1–8	SOUT4, SOUT3, –, VS_IN, SOUT2, SOUT1, AIN2, AIN1
Terminal de entradas	1–8	AIN3, AIN4, VS_IN, –, DI1, DI2, DI3, DI4
Filas de masa	1–8	GND

[VERIFICAR: comprobar el orden de las bornas sobre la placa fabricada]

Apéndice B

Especificación del protocolo binario

Formato común: 0xAA 0x55 LEN carga_útil[LEN] CRC8. Enteros en *little-endian*. CRC-8 con polinomio 0x07 y valor inicial 0x00 sobre la carga útil. Los desplazamientos se dan en bytes dentro de la carga útil.

B.1. Paquete 0x02: telemetría por lotes

Tabla B.1: Cabecera del lote (14 bytes).

Despl.	Campo	Tipo	Descripción
0	packet_id	u8	0x02
1	base_t	u32	millis() de la primera muestra del lote
5	bridge	u16	Lectura bruta del ZSC31014 (14 bits)
7	lc_status	u8	Bits de estado de la última lectura
8	flags	u8	b0: célula válida; b1: servo conectado; b2: configuración aplicada
9	error	u8	Código de error (tabla B.4)
10	servo_id	u8	Dirección Modbus en uso
11	rpm_status	u8	Resultado de la lectura de velocidad (tabla B.5)
12	trq_status	u8	Resultado de la lectura de par
13	n	u8	Número de muestras (1–8)

Tabla B.2: Muestra (14 bytes), repetida n veces.

Despl.	Campo	Tipo	Descripción
+0	dt_ms	u16	Milisegundos desde base_t
+2	rpm	i16	Velocidad del motor (registro 16385)
+4	current_x10	i16	Par (registro 16387), décimas de u. a.
+6	ref_cmd	i16	Consigna de velocidad escrita (rpm)
+8	base_read	i16	Lectura filtrada menos tara (cuentas)
+10	io	u8	b0: final A; b1: final B
+11	state	u8	Estado de la máquina (tabla B.3)
+12	work_us	u16	Tiempo de trabajo de la vuelta (μ s, saturado a 65535)

B.2. Paquete 0x03: estado de *homing* (11 bytes)

0	packet_id	u8	0x03
1	pos_mrev	i32	Posición $\times 1000$
5	range_mrev	i32	Recorrido A $\rightarrow$ B medido $\times 1000$
9	home_phase	u8	0 reposo, 1 buscando A, 2 buscando B, 3 yendo al origen, 4 completado, 5 fallo
10	flags	u8	b0: recorrido válido

B.3. Paquete 0x04: resultado de la célula (13 bytes)

0	packet_id	u8	0x04
1	req	u8	Orden respondida (0x09, 0x0A o 0x0B)
2	result	u8	Resultado (tabla B.6)
3	gain	u8	Ganancia grabada (código 0–7)
4	offset	u8	Cero del A/D grabado (0–15)
5	cfg_before	u16	B_Config antes
7	cfg_after	u16	B_Config después
9	ack	u8	Primer byte devuelto por el chip
10	diag	u8	b0: el reinicio corta; b1: hubo ACK a Start_CM; b7–b4: intento del barrido
11	cfg1	u16	ZMDI_Config1

B.4. Paquete 0x05: estado del ciclo (14 bytes)

0	packet_id	u8	0x05
1	phase	u8	0 reposo, 1 hacia B, 2 volviendo a A, 3 completado, 4 abortado
2	done	u16	Repeticiones completadas
4	target	u16	Repeticiones pedidas (0 = sin fin)
6	pos_mrev	i32	Posición $\times 1000$
10	target_mrev	i32	Posición de B $\times 1000$

B.5. Órdenes

Orden	Carga útil	Notas
INIT	01 [addr]	addr opcional, 1–247
STOP	02	
SET_PARAM	03 id vv vv	valor i16
MOVE_A / MOVE_B	04 / 05	
SHUTDOWN	06	
SCAN	07	bloqueante
HOME	08	
LC_PROGRAM	09 bb bb cc cc	B_Config u16, ZMDI_Config1 u16
LC_QUERY	0A	
LC_HOLD	0B park	0: alta impedancia; 1: líneas a masa
CYCLE_START	0C mm tt tt vv vv rr rr	mm = 0: B es el final; 1: B es una posición. Objetivo i16 (centi-rev), velocidad i16, repeticiones u16
CYCLE_STOP	0D	

Atajos de terminal (sólo con el decodificador en reposo): **i** INIT, **a** MOVE_A, **b** MOVE_B, **x** STOP, **h** HOME, **q** SHUTDOWN.

B.6. Tablas de códigos

Tabla B.3: Estados de la máquina.

0	IDLE (servo sin habilitar)
1	SCANNING
2	CONFIGURING
3	STOPPED (habilitado, consigna 0)
4	MOVING_A
5	MOVING_B
6	ERROR
7	HOMING
8	PROGRAMMING (célula)

Tabla B.4: Códigos de error.

0x00	Sin error
0x01	Célula no encontrada
0x02	Fallo en el escaneo del servo
0x03	Fallo al configurar el servo
0x04	Límite de fuerza

Tabla B.5: Resultados de ModbusMaster.

0x00	Correcto
0x01–0x04	Excepción del esclavo
0xE0	Dirección de esclavo incorrecta
0xE1	Función incorrecta
0xE2	Tiempo de espera agotado
0xE3	CRC incorrecto

Tabla B.6: Resultados de las órdenes de la célula.

0	Nunca se ha intentado
1	Grabado y verificado tras un ciclo de alimentación
2	Ya estaba así: no se ha escrito la EEPROM
3	Rechazado: el eje no está parado
4	Rechazado: bus ocupado
5	El chip se reinicia pero no entra en modo comando
6	Escrito, pero la relectura no coincide
7	Sólo lectura de la configuración
8	Sigue alimentada con el reinicio activo: hay otro camino de alimentación
9	La célula no contesta: sin tensión o sin bus
10	Reinicio mantenido para medir

Apéndice C

Registros Modbus del accionamiento

Dirección = 256·grupo + índice (índice en hexadecimal). Esclavo 1, 115200 baudios, 8N1.
[**VERIFICAR:** contrastar denominaciones y unidades con el manual del accionamiento]

Tabla C.1: Registros utilizados por el firmware.

Dirección	Parámetro	Acceso	Descripción	Valor escrito
0	C00.00	E	Modo de control (1 = velocidad, 2 = par)	1
5	C00.05	E	Nivel de rigidez	10
16	C00.10	E	Selección de resistencia de frenado (1 = externa)	1
17	C00.11	E	Potencia de la resistencia de frenado	1000
18	C00.12	E	Valor óhmico de la resistencia de frenado	235
19	C00.13	E	Disipación de la resistencia de frenado	30
801	C03.21	E	Consigna de velocidad (rpm, con signo)	cada periodo
802	C03.22	E	Aceleración	5
804	C03.24	E	Deceleración	5
1041	C04.11	E	Habilitación del servo (0/1)	0 → 1
16385	0x4001	L	Velocidad del motor (rpm)	–
16387	0x4003	L	Par del motor (décimas de u. a.)	–

Registros usados en versiones anteriores (modo par): 833 (consigna de par, C03.41), 839 y 840 (límites de par), 1542 (C06.06, apantallamiento de STO) y 1576 (C06.28, retardo de desconexión tras STO).

Apéndice D

Formato de los ficheros de ensayo

Cada ciclo se graba en `cycles/AAAAMMDD-HHMMSS_ciclo.csv`, codificado en UTF-8. Las líneas que empiezan por `#` forman la cabecera, con las constantes en pares `clave=valor`; después viene una línea de nombres de columna y una fila por muestra.

```
# ensayo 2026-09-15 18:13:46
# lc1_cuentas_por_N=0
# repeticiones_pedidas=1 B=1.000 rev (posicion)
# lc1_base ya lleva restada la tara del firmware; lc1_raw es cruda
t_ms,t_rel_s,rep,fase,pos_rev,lc1_raw,lc1_base,rpm,par_x10,ref_rpm,lim_a,lim_b,estado,
work_us
```

Listado D.1: Comienzo del fichero 20260915-181346_ciclo.csv.

La versión actual del servidor escribe además `lc1_cero_N` (fuerza en el punto de tara) y `lc1_int16` en la segunda línea.

Tabla D.1: Columnas del fichero de ensayo.

Columna	Unidad	Descripción
t_ms	ms	Reloj del microcontrolador
t_rel_s	s	Tiempo desde la primera muestra del ensayo
rep	–	Repetición en curso (desde 1)
fase	–	<code>hacia_B</code> o <code>volviendo_a_A</code>
pos_rev	rev	Posición del eje del motor integrada por el servidor
lc1_raw	cuentas	Lectura bruta del ZSC31014 (una por lote)
lc1_base	cuentas	Media móvil de 8 lecturas menos la tara del firmware
rpm	rpm	Velocidad medida por el accionamiento
par_x10	0,1 u. a.	Par medido por el accionamiento
ref_rpm	rpm	Consigna de velocidad
lim_a, lim_b	0/1	Estado de los finales de carrera
estado	–	Estado de la máquina (tabla B.3)
work_us	µs	Tiempo de trabajo de la vuelta del lazo

Conversión a fuerza: $F \text{ [N]} = \text{lc1_base}/k + F_0$, con $k = \text{lc1_cuentas_por_N}$ y $F_0 = \text{lc1_cero_N}$.

Apéndice E

Manual de uso

E.1. Instalación

E.1.1. Firmware

1. Instalar Arduino IDE y añadir en *Preferencias* el gestor de placas de Arduino-Pico:
https://github.com/earlephilhower/arduino-pico/releases/download/global/package_rp2040_index.json
2. Instalar el paquete *Raspberry Pi Pico/RP2040/RP2350* y la biblioteca *ModbusMaster* desde el gestor de bibliotecas.
3. Abrir *firmware_posicion_v2/firmware_posicion_v2.ino* y seleccionar la placa *Raspberry Pi Pico 2*.
4. Conectar la Pico con el botón BOOTSEL pulsado y cargar el programa.

E.1.2. Aplicación de supervisión

```
python3 -m venv .venv
source .venv/bin/activate
pip install fastapi "uvicorn[standard]" pyserial
python web_monitor.py # o: python web_monitor.py /dev/ttyACMO
```

En Linux, la cuenta de usuario debe pertenecer al grupo `dialout` para acceder al puerto. La interfaz se abre en `http://localhost:8080`.

E.2. Procedimiento de ensayo

1. **Comprobaciones previas:** masa colgada, cable sin cocas, finales de carrera en reposo (LED apagados en la interfaz), zona de movimiento despejada.
2. **Inicializar:** pestaña *Servo*, dirección 1, botón *Init*. El estado pasa a STOPPED y el indicador del servo a conectado.
3. **Homing:** fijar la velocidad de *homing* y el desplazamiento del origen, pulsar *Aplicar parámetros* y después *Home*. Al terminar se muestra el recorrido A→B.
4. **Tara y calibración:** con la masa en la posición de referencia, pulsar *Tara*; introducir en la pestaña *Célula 1* la calibración (cuentas/N) y la fuerza real en el punto de tara, y aplicar.

5. **Ciclo:** en la pestaña *Ciclo A↔B*, elegir el punto B, la velocidad y las repeticiones, y pulsar *Iniciar ciclo*. La grabación empieza y termina sola.
6. **Análisis:** botón *Ver ensayos*; abrir el fichero y, si se desea, desplegar *superponer curva teórica* e introducir la geometría.
7. **Fin:** *Shutdown* para deshabilitar el accionamiento.

E.3. Programación de la célula de carga

Con el eje parado, abrir *Configuración avanzada* en la pestaña de la célula, elegir la ganancia y el cero del A/D y pulsar *Programar*. El eje queda bloqueado cerca de un segundo. El resultado aparece debajo; si es “sigue alimentada con el reset abajo”, usar *Mantener reset* y medir la tensión de alimentación de la placa Click, que debe caer a 0 V.

E.4. Diagnóstico

Síntoma	Causa probable y acción
Servo desconectado, lecturas con código 0xE2	Tiempo de espera Modbus: revisar el cableado A/B, la masa común, la dirección y la velocidad del accionamiento. Probar con <code>test_modbus_scan</code> .
Código 0xE3	CRC incorrecto: ruido o polaridad A/B invertida.
Estado ERROR con código 0x03	Tres fallos de configuración: el accionamiento no acepta las escrituras (¿alimentado?, ¿alarma activa?).
Final de carrera siempre activo	Contacto abierto o cable roto (entrada NC).
Célula no válida	Sin lecturas frescas en 150 ms: revisar la placa Click y el bus I ² C con <code>test_loadcell</code> .
La interfaz no muestra datos	Comprobar el puerto en la barra superior y el fichero <code>web_monitor.log</code> .

E.5. Precauciones

- Las protecciones son de software: mantener las manos fuera del recorrido con el servo habilitado y tener a mano el corte de alimentación del accionamiento.
- No programar la célula ni usar *Mantener reset* con carga en movimiento (el firmware lo impide, pero la medida de fuerza queda interrumpida).
- El servidor acepta órdenes de cualquier equipo de la red: usarlo en una red de confianza.

Apéndice F

Estructura del repositorio

Ruta	Contenido
Firmware principal	
<code>firmware_posicion_v2/</code>	Firmware final (Pico 2): Modbus, célula, <i>homing</i> , ciclos, protocolo. <code>i2c_bus.h</code> : árbitro del bus I ² C.
<code>firmware_posicion/</code>	Versión anterior en modo velocidad A→B.
<code>firmwarerasp/</code>	Primera versión para Pico 2, doble núcleo, modos par y velocidad.
<code>firmware/</code>	Versión para XIAO ESP32-S3 con regulador de fuerza en modo par.
Programas de prueba	
<code>pcbtest/test_analog,</code> <code>test_din,</code> <code>test_digital_outputs</code>	Entradas analógicas, entradas digitales y salidas de potencia.
<code>test_limit_switches/</code>	Finales de carrera.
<code>test_rs485*/,</code>	Bucle local PIO, transmisión, recepción y comunicación Modbus.
<code>test_modbus_scan/</code>	ZSC31014 y lectura dual con NAU7802.
<code>test_loadcell*/</code> <code>test_servo_modbus.py</code>	Barrido Modbus desde el PC con adaptador USB-RS485.
Ordenador	
<code>web_monitor.py</code>	Servidor FastAPI, puente serie-WebSocket y grabación.
<code>static/index.html</code>	Página de supervisión.
<code>static/ensayos.html</code>	Visor de ensayos con modelo teórico.
<code>leer_ciclo.py</code>	Resumen de ensayos en consola.
<code>monitor.py,</code> <code>monitor_posicion.py</code>	Monitores de terminal de versiones anteriores.
<code>cycles/</code>	Ficheros de ensayo (CSV).
Hardware y documentación	
<code>tfg_adaptador/</code>	Proyecto KiCad de la placa (revisión A) y copias de seguridad.
<code>image (1).png</code>	Esquema de la revisión B.
<code>README.MD</code>	Notas sobre la UART de la Pico y el doble núcleo.

Ruta	Contenido
<code>memoria/</code>	Esta memoria ($\text{\LaTeX}$), figuras y <code>generar_figuras.py</code> .
<code>old/</code>	Pruebas iniciales en modo par.